ЛАБОРАТОРНАЯ РАБОТА №5.
ОРГАНИЗАЦИЯ РЕПОЗИТОРИЯ В СИСТЕМЕ УПРАВЛЕНИЯ ВЕРСИЯМИ GIT

1. ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

Цель: получение практических навыков работы с распределенной системой управления версиями Git и сервисом для хранения, совместной работы и управления кодом.

Для достижения поставленной цели требуется решить следующие **задачи:**

1. Создать пространство проекта на сервисе для хранения, совместной работы и управления кодом, основанной на системе контроля версий git (например, GitHub, GitVerse, GitLab, SourceCraft).
2. Разработать три ветви проекта.
3. Выполнить форк внешнего репозитория.
4. Реализовать индивидуальный проект по использованию сервиса согласно методическим указаниям.

Структура методических рекомендаций

В разделе 1 представлены цели и задачи практического задания. Также дается пояснение выбора проекта разработки.

В разделе 2 показаны критерии оценивания. Внимательно изучите этот раздел.

В разделе 3 представлены назначение систем управления версиями и рассмотрены возможности системы под управлением Git. В этом же разделе показаны возможности онлайн-сервиса GitHub. Обратите внимание, что вы также можете использовать GitLab или отечественные аналоги GitVerse или SourceCraft.

В разделе 4 представлено выполнение упражнений, особенно полезно для тех, кто не использовал этот сервис раньше.

В разделе 5 представлено пример задания на индивидуальный проект (обязательно ознакомьтесь с разделом, чтобы выполнить задание).

В конце методических указаний представлен список литературы и фрагмент примера выполнения отчета в Приложениях.

Общие рекомендации

Внимательно прочитайте методические указания и выполните упражнения.

Если вы знакомы и работаете с GitHub (GitLab и др.), то можно сделать несколько скриншотов из своего репозитория. Достаточно продемонстрировать факт того, что вы умеете работать с такими сервисами.

Если вы мало знакомы или не имеете своего репозитория, то выполните все упражнения. Скриншоты выполнения каждого упражнения в отчете показывать не требуется. Достаточно 5-6 скриншотов, которые подтверждают, что вы создали репозиторий, заполнили его и научились создавать ветви. Также нужно форкнуть любой выбранный репозиторий. Выбор репозитория, который планируете перенести осуществите самостоятельно, например используя список самых интересных репозиториев [1].

Обратите внимание, что платформа GitHub может обновить интерфейс и расположение функциональных элементов может немного измениться и отличаться от того, что представлено на скриншотах.

Если справиться с заданием не удается, то в отчете покажите на скриншотах, на каком этапе произошли критические проблемы, в выводе попробуйте дать им объяснения.

После выполнения всех упражнений переходите к выполнению **индивидуального проекта** по применению Git: реализация веб-страницы, которую вы создали **в лабораторной работе №4**. В методических указаниях также показаны примеры, что необходимо сделать в рамках этой работы (**раздел 5**). Обязательно ознакомьтесь с этим разделом.

При выполнении индивидуального задания рекомендуется пользоваться VS Code, как показано в примерах. Но можно использовать любые другие IDE, поддерживающие GIT по согласованию с вашим преподавателем практики. **Обязательно указать в отчёте, какой сервис вы будете использовать.**

При выполнении задания преподаватель практики не сопровождает пошагово процесс его реализации. В его обязанности входит фиксация темы и проверка полноты выполнения. Поэтому выбирайте тему и задачи выполнения в зависимости от уровня сформированных компетенций по применению Git.

Рекомендации по использованию агентов.

1. Вы можете выбрать любого доступного вам агента.
2. Если https://sourcecraft.dev работает нестабильно, попробуйте установить GigaCode - https://gitverse.ru/features/gigacode/install/?onboarding=true
3. Также вы можете попробовать установить Cline или Copilot и использовать их.

Последовательность выполнения задания

1. Зарегистрироваться на сервисе, который вы выбрали (например, Github).

2. Выполнить общие упражнения: создать не менее трех ветвей проекта, в каждой из которых сохранить файлы разных форматов: код программы, выполненный на любом языке программирования, или текстовые файлы, например, отчеты по лабораторным работам, графические материалы и т.д. Каждое изменение (новую ветвь, дополнение репозитория) фиксировать в системе контроле версий. Если уже работали с git – переходите сразу к пункту 3.

3. Скопировать любой программный проект, который хранится на GitHub, например, библиотеку pytorch, в свой удаленный репозиторий.

4. Выполнить **индивидуальный проект** (используйте сайт, созданный в лабораторной работе №4), предварительно зафиксировав тему у преподавателя. При выполнении индивидуального задания используйте последовательность действий, показанную в методических рекомендациях (***см. раздел 5, стр. 24***). Вам необходимо дополнить сайт, который вы создавали в предыдущей работе, при этом обязательно необходимо:

4.1 Выложить свой проект в **публичный** репозиторий на github.

4.2 Использовать агента для внесения изменений в сайт, предоставлять скриншоты промтов для изменений, а также самих изменений сайта (было/стало), скриншоты с github. Вы можете выбрать любого агента.

4.3 В вашем проекте должно быть создано **не менее 3 веток проекта** (подтвердите скриншотами и текстовым описанием).

4.4 В вашем проекте необходимо добавить или изменить как минимум **три фотографии** на сайте (добавьте скриншоты промта, рабочей папки с изображениями, текстовые комментарии).

4.5 В вашем проекте необходимо сделать как минимум одно объединение ветки проекта с main (подтвердите скриншотами и текстовым описанием).

4.6 Вам необходимо опубликовать ваш сайт с помощью **GitHub Pages**, предоставить ссылку на проект в отчёте. Проверьте, что ссылка доступна и работает.

5. Выполнить отчет в соответствие с требованиями к структуре и наполнению.

6. Перечитайте раздел 2, в котором указаны критерии оценивания. Проверьте, что требования соблюдены.

Примерная структура отчета по практической работе

Цель работы.

1) Созданный аккаунт на GitHub с календарем активностей (рис.1.1-1.2).

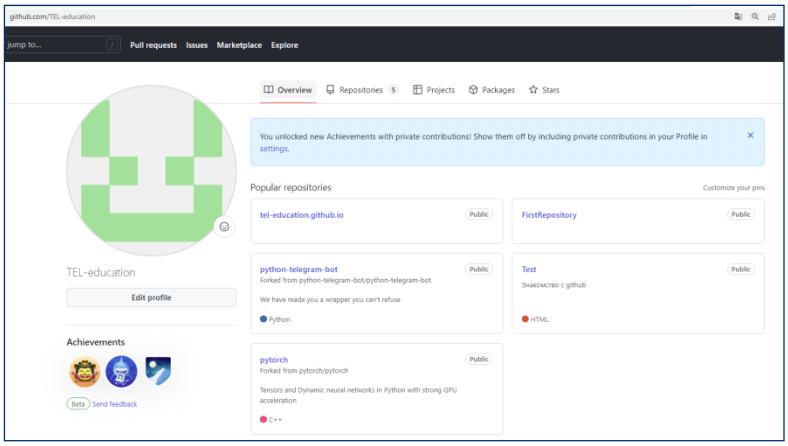


Рисунок 1.1 - Пример аккаунта



Рисунок 1.2 - Активность аккаунта

2) Скриншоты трех ветвей проекта на удаленном сервере GitHub (рис.1.3-1.4).

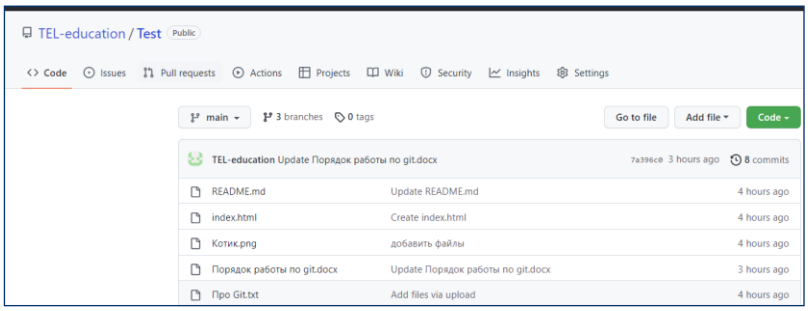


Рисунок 1.3 - Ветвь main

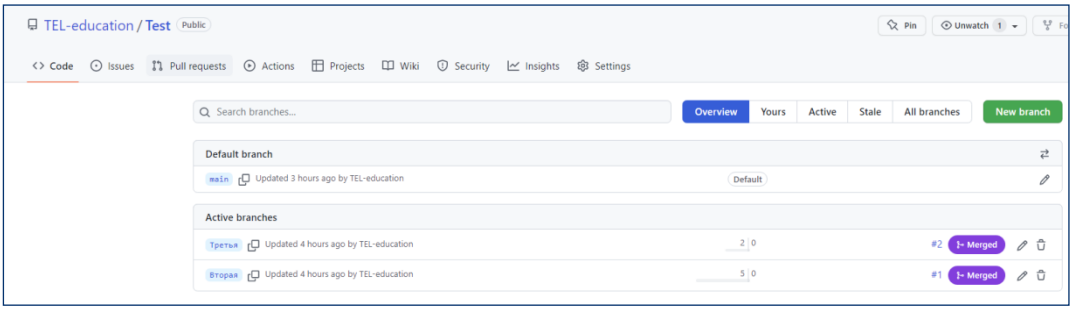


Рисунок 1.4 - Ветви проекта

3) Скриншот копии выбранного проекта с полезными библиотеками (рис.1.5).

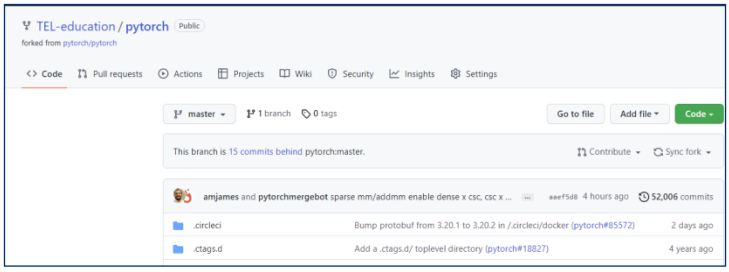


Рисунок 1.5 - Пример скриншота копии библиотеки pytorch

4) Ссылка на удаленный репозиторий на GitHub, реализованный в ходе выполнения практической работы.
5) Текстовые комментарии и скриншоты индивидуального проекта. **Для выполнения этой части используйте рекомендации из методических указаний.**
6) Вывод, в котором в расширенном формате необходимо раскрыть цель выполненной работы, описать знания и навыки, полученные в процессе ее выполнения, а также возникшие проблемы и пути их решения.
7) Список использованных источников.

2. КРИТЕРИИ ОЦЕНИВАНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

Оценивание практической работы выполняется по следующим критериям. За выполнение работы выставляются баллы, которые потом переводятся в оценку в случае дифференцированного зачета или зачет/не зачет в случае зачета без оценки.

Таблица 2.1 – Критерии оценивания практической работы

№	Критерий	Балл
1	Цели и задачи работы	Выбрана и дано описание – 1 балл Не выбрана -0 баллов
2	Дана ссылка на репозиторий проекта	Да – 1 балл Нет – 0 баллов
3	Показаны ветви индивидуального проекта, не менее 3 ветвей	Да – 1 балл Нет – 0 баллов
4	Показан форк проекта	Да – 1 балл Нет – 0 баллов
5	В отчете скриншоты представлены в таком количестве, чтобы преподаватель увидел файлы проекта, три ветви проекта и форк внешнего проекта, а также индивидуальный проект	Да – 2 балл Нет – 0 баллов
6	Присутствует текстовое пояснение того, что изображено на скриншотах	Да – 2 балл Нет – 0 баллов
7	Индивидуальный проект с описанием и скриншотами	Да – 3 балла Нет – 0 баллов
8	Дана ссылка на сайт на github	Да – 1 балл Нет – 0 баллов
9	Используется агент для внесения изменения в сайт	Да – 1 балл Нет – 0 баллов
10	На сайт были изменены или добавлены собственные изображения из рабочей папки с помощью агента	Да – 1 балл Нет – 0 баллов
11	В выводе сформулированы задачи, которые решены, показано назначение СУВ	Да – 1 балл Нет – 0 баллов
Максимально возможный балл		**15**

Таблица 2.2 – Перевод баллов в оценку

Баллы	Оценка	Зачет/не зачет
Менее 7 баллов	неудовлетворительно	Не зачет
7-10 баллов	удовлетворительно	Зачет
11-12	хорошо	
13-15	отлично	

3. НАЗНАЧЕНИЕ СИСТЕМ УПРАВЛЕНИЯ ВЕРСИЯМИ

При работе над программным проектом каждый из команды разработчиков должен иметь доступ ко всем файлам. Поэтому возникает необходимость организации виртуального хранилища программных компонент с **системой контроля версий.**

Существуют множество систем управления версиями. Условно их можно разделить три их главные группы:

- в соответствии с расположением репозитория: централизованные и распределенные;
- в соответствии с методами проверки слияния и передачи кода: блокирующие, использующие слияние до фиксации и выполняющие фиксацию до слияния;
- системы управления версиями могут выполнять небольшие операции или операции с файлами.

Работу с системами контроля версий рассмотрим на примере распределенной системы управления версиями Git[1].

Программные средства данной системы позволяют сохранить изменения в файл или набор файлов в процессе их модификации и при необходимости вернуться к конкретной версии файла. Если над проектом работают несколько человек, то каждому из них обеспечивают доступ для совместной работы над файлом. Каждое внесенное изменение фиксируют, поэтому возникает **многоверсионность** разрабатываемого программного продукта. Недостатком Git можно считать то, что он имеет интерфейс командной строки. Чтобы упростить взаимодействие разработчиков с этой системой, был создан графический интерфейс GitHub[2].

GitHub (https://github.com/) – крупнейший веб-сервис для хостинга IT-проектов и их совместной разработки. Веб-сервис основан на системе контроля

[1] https://git-scm.com/
[2] https://github.com

версий Git. Ознакомиться с документацией по работе с GitHub можно на сайте GitHub Help (https://help.github.com/en) .

Также GitHub содержит миллионы открытых проектов, которые можно использовать бесплатно.

Git позволяет отслеживать:

- какие файлы были изменены;
- какие строки были добавлены или удалены;
- кто и когда внёс изменения;
- какие изменения относятся к определённой версии проекта;
- какие версии проекта существовали ранее;
- какие изменения были объединены между различными направлениями разработки.

Существуют также российские аналоги GitHub:

GitVerse (https://gitverse.ru/) - российская платформа для хостинга IT-проектов. Поддерживает Git, CI/CD, имеет встроенный AI-ассистент GigaCode.

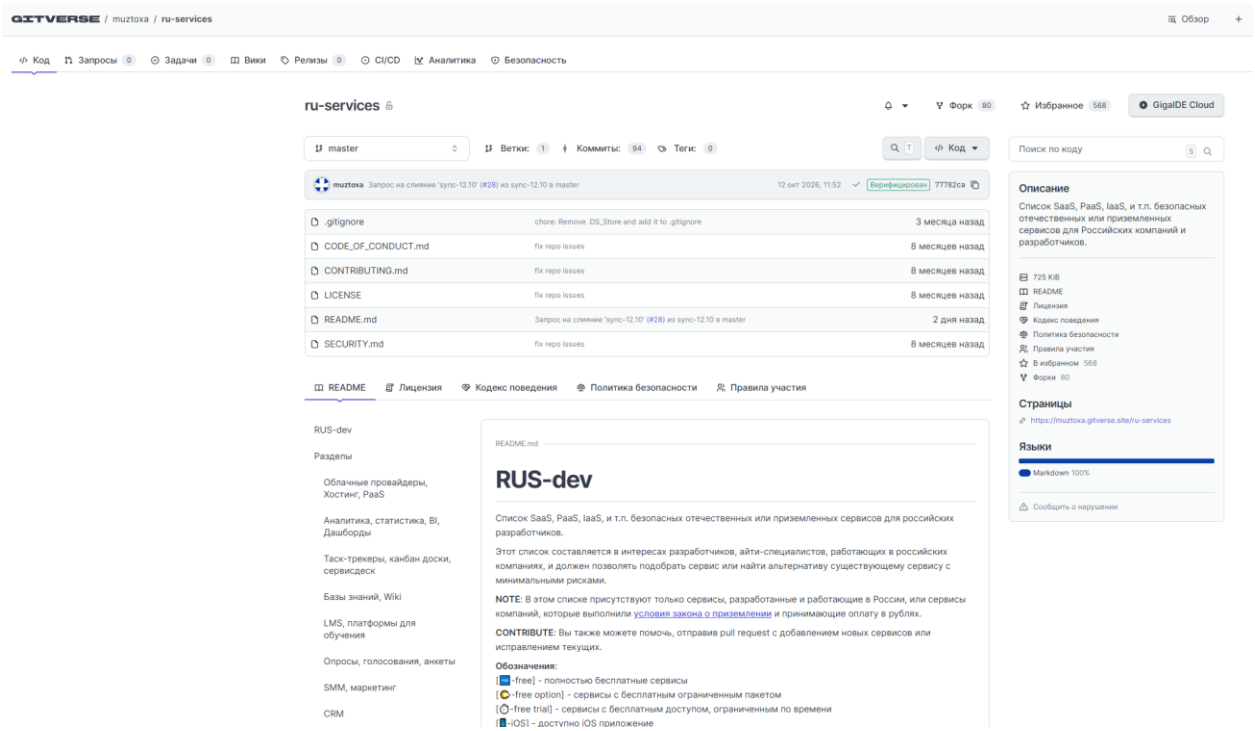


Рисунок 2.1 – Пример репозитория в GitVerse

SourceCraft (https://sourcecraft.dev/) - российская платформа для совместной разработки от Яндекс. Поддерживает Git, ревью кода, CI/CD.

Основная задача Git – **управление историей изменений проекта**.

Git и GitHub связаны, при этом можно использовать Git без GitHub, используя локальный репозиторий, не отправляя проект в интернет.

Репозиторий – это хранилище проекта, управляемое Git.

Рабочая директория – это обычные файлы проекта, с которыми разработчик работает (например, непосредственно в VS Code).

Рассмотрим некоторые команды в git.

Clone – клонировать репозиторий с GitHub.

Init – инициализировать новый локальный репозиторий.

Add – добавить измененные файлы в индекс (stage).

Commit – сохранить изменения в истории проекта, это снимок проекта в определённый момент времени. Каждый commit имеет уникальный идентификатор, автора, дату, информацию об изменениях. Хорошие сообщения commit должны кратко описывать выполненную работу. Также перед commit изменения должны быть сохранены в файле.

Работы с git можно осуществлять через терминал, либо с помощью интерфейса (например, в VS Code).

Ветка (branch) – отдельная линия разработки проекта. Основная ветка обычно называется main. Ветки позволяют одновременно разрабатывать различные функции. Один разработчик может работать над авторизацией, другой над дизайном, третий исправлять ошибку.

Когда разработка функции закончена, изменения можно объединить с main. Иногда Git не может автоматически объединить изменения, если была изменена одна и та же информация, возникает merge conflict.

Push – отправить локальные commit в удалённый репозиторий (на GitHub).

Pull – обратная операция, получает изменения из удаленного репозитория (из GitHub и соединяет их с текущей локальной веткой).

Онлайн-сервис GitHub позволяет создавать репозитории проектов без глубоких знаний программирования. Все требуемые команды git будут сгенерированы автоматически на основе действий пользователя. В репозитории проекта можно хранить любые файлы. Текстовые файлы можно создавать непосредственно в репозитории с помощью языка Markdown. Там же можно их изменять. Если возможностей текстового редактора не хватает для внесения изменений, то файлы скачивают на персональный компьютер. В этом случае на компьютере у пользователя должна быть установлена программа GitHub Desktop. Её пространство полностью синхронизировано с онлайн-пространством GitHub. При внесении изменений в файлы они будут автоматически переданы в облачную версию.

4. ВЫПОЛНЕНИЕ УПРАЖНЕНИЙ

Упражнение 1. Создание аккаунта GitHub.

1.Зарегистрируйтесь на сервисе GitHub (`https://github.com/`). Для этого перейдите на сайт и нажмите кнопку Sign up (рис.3.1). В открывшемся меню введите параметры учетной записи: имя пользователя; адрес электронной почты; пароль.

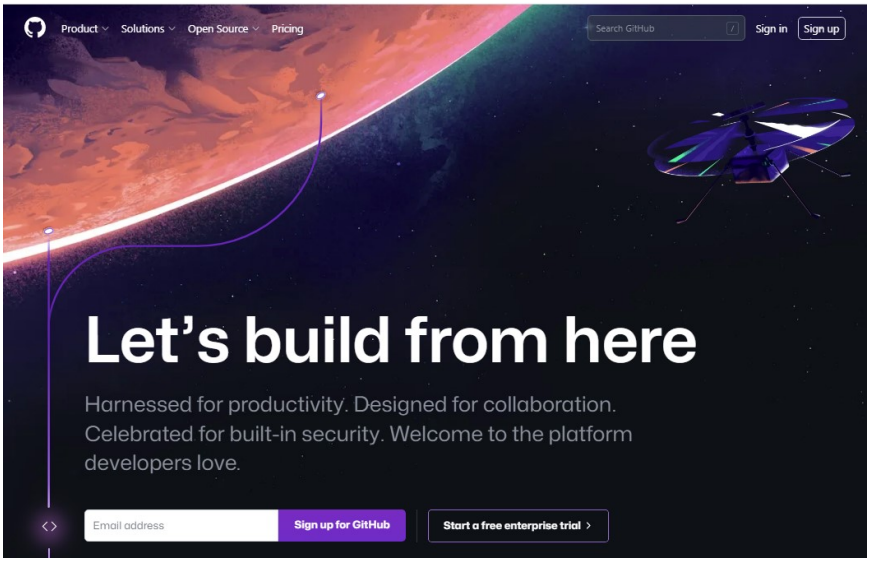


Рисунок 3.1 - Регистрация на сервисе GitHub

2. Полное имя аккаунта посмотрите на вкладке Профиль, `например TEL-education.github.io` (рис. 3.2).

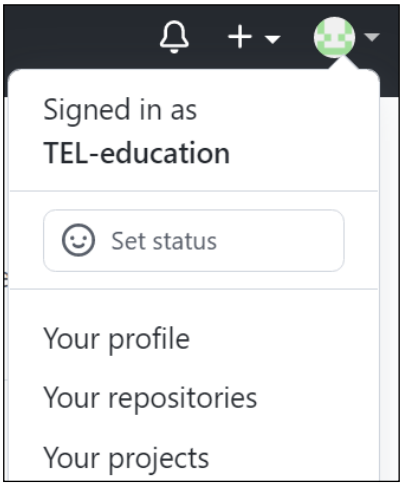


Рисунок 3.2 - Проверка имени аккаунта

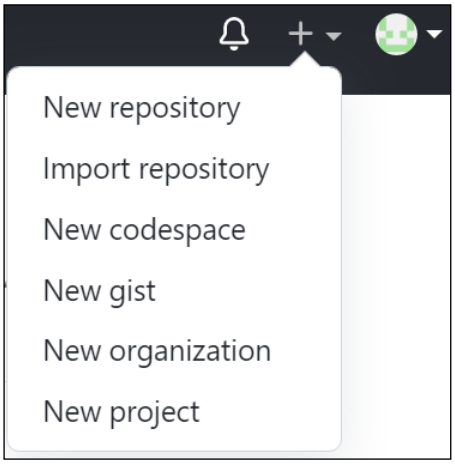


Рисунок 3.3 - Создание нового репозитория New repository

3. После регистрации создайте свой первый репозиторий. Для этого нажмите на знак Плюс в правом верхнем углу окна GitHub. Назовите репозиторий, например

Space и нажмите кнопку **Create Repository** (рис. 3.3-3.4). Все названия проектов на GitHub необходимо писать латинским алфавитом. Названия файлов в рассмотренных ниже примерах приводятся на русском языке, чтобы упростить понимание процесса взаимодействия с удаленным репозиторием. В описание репозитория внесите: «Знакомство с GitHub».

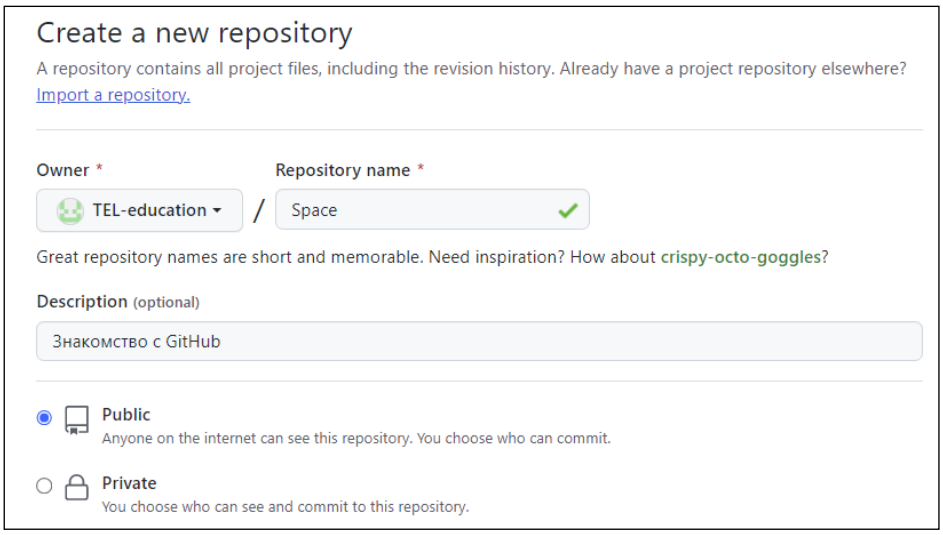


Рисунок 3.4 - Создание репозитория на сервисе GitHub

Упражнение 2. Копирование файлов проекта в репозиторий на GitHub

1. Подготовьте на персональном компьютере не менее пяти файлов для копирования их в репозиторий (рис.3.5). Файлы могут иметь любой формат и размер и быть созданы в любых программных средах.

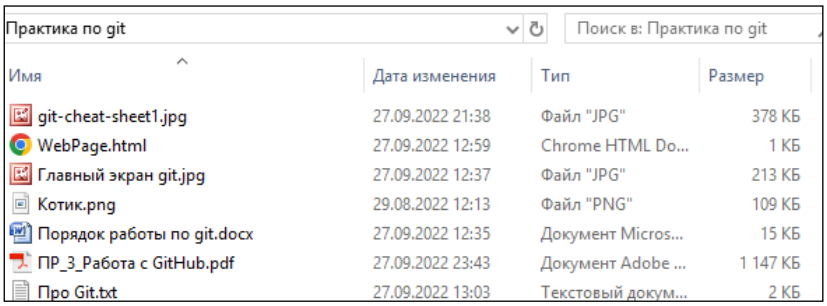


Рисунок 3.5 - Подготовленные для копирования файлы

2. После создания репозитория будет доступно окно для выбора дальнейших действий пользователя (рис.3.6). Выберите `uploading an existing file` (загрузку внешних файлов).

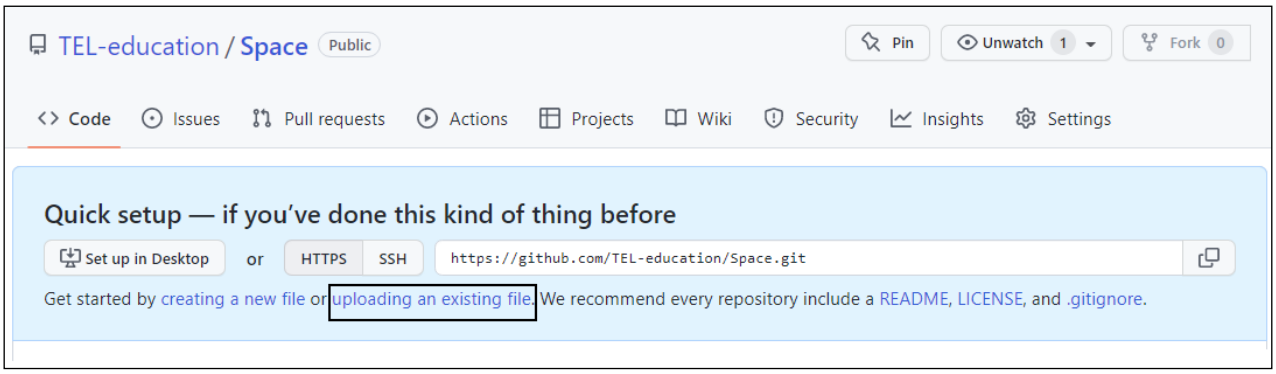


Рисунок 3.6 - Выбор опции uploading an existing

3. Затем перетяните в открывшееся окно два любых файла для копирования в основную ветвь проекта. Основная ветвь проекта по умолчанию названа `main`. Обратите внимание, на автоматически сгенерированную команду `Add files via upload`, которая появилась в разделе *Commit changes* (рис.3.7).

Добавьте описание, объясняющее, какое именно изменение было сделано. В рассматриваемом примере: «Копирование двух файлов проекта Space». Описания фиксируют историю изменений в коде проекта, чтобы другие участники проекта могли увидеть изменения и понять их назначение. Нажмите на кнопку **Commit changes** и завершите первый коммит.

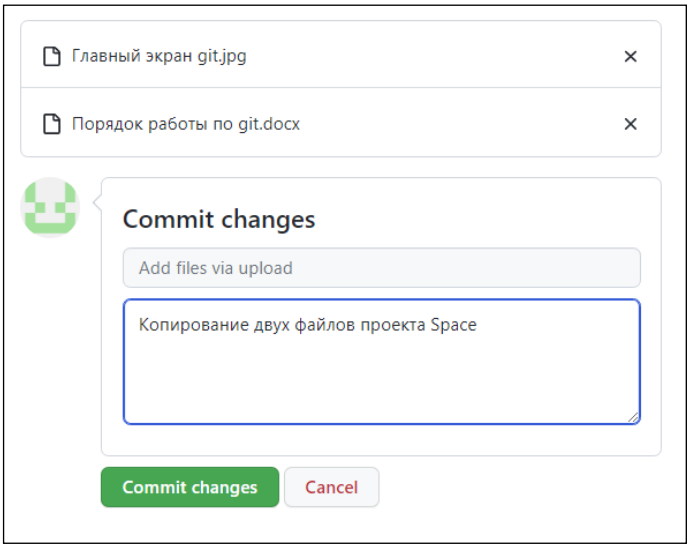


Рисунок 3.7 - Добавление файлов проекта в репозиторий

2. Результат копирования файлов с персонального компьютера в облачное хранилище показан на рис.3.8.

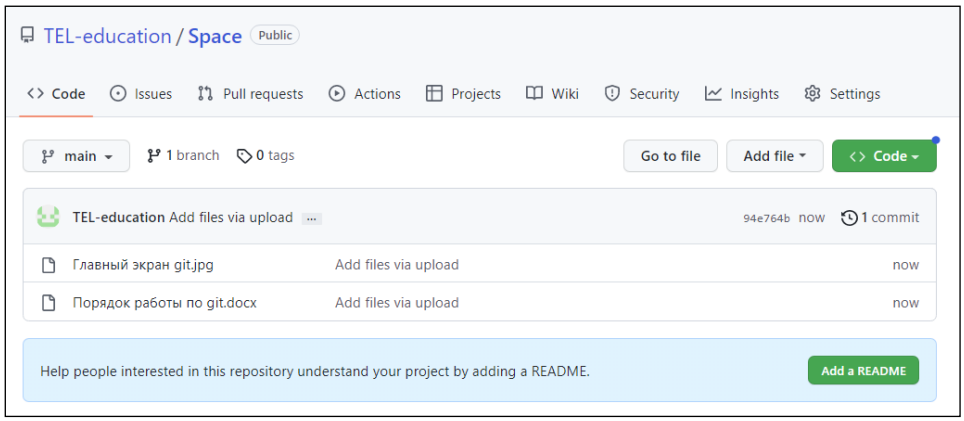


Рисунок 3.8 - Ветви проекта Обучение в репозитории TEL-education

На этом этапе в репозитории **TEL-education** создан проект **Space**, в котором присутствует только одна ветвь проекта `1branch` (первая ветвь) под названием `main,` в которой сохранены два файла.

3. В каждом проекте предусмотрено его описание в файле `README.md`. Нажмите на кнопку **Add a README** для перехода в область для создания файла.

4. В текстовую область вкладки меню *Edit new file* (рис.3.9) уже внесено описание репозитория: «Знакомство с GitHub». В области Commit new file обратите внимание на команду Create README.md. Внесите описание коммита: «Добавление файла README».

5. Выполните слияние с основной ветвью проекта `main branch`, нажав на кнопку **Commit new file**.

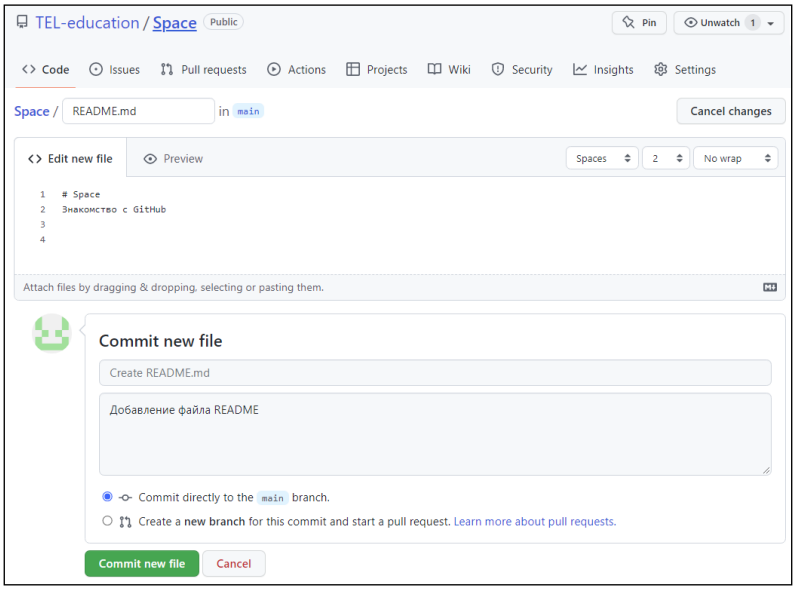


Рисунок 3.9 - Подготовка нового коммита по добавлению файла README.md.

После этого шага в репозитории Space сохранены три файла (рис.3.10).

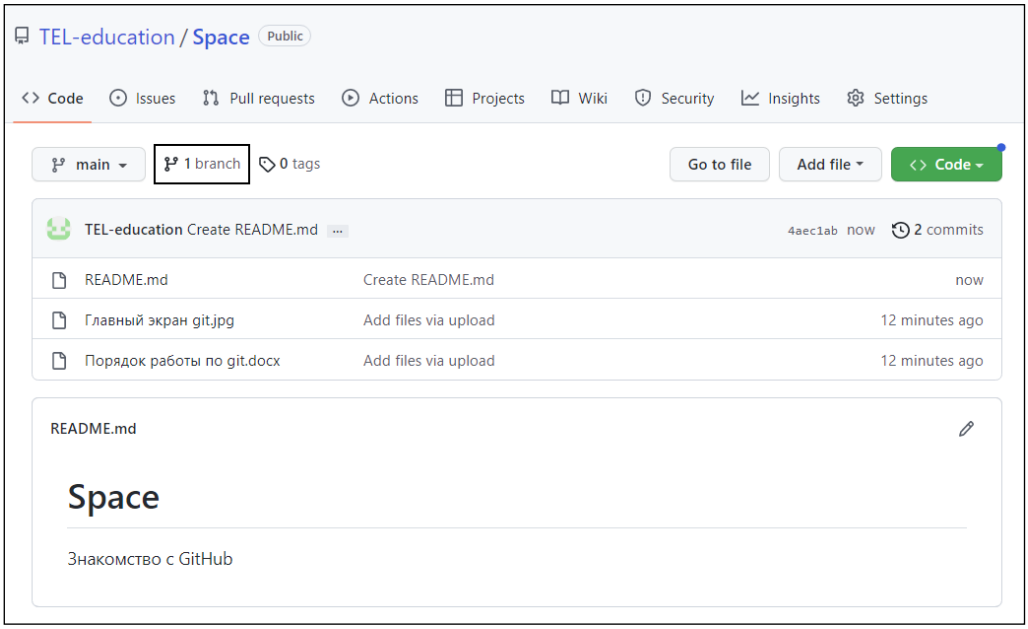


Рисунок 3.10 - Количество файлов в ветви main после первого коммита

Упражнение 3. Добавление второй ветви проекта Space

1. Кликните по пиктограмме 1 branch (рис. 3.10).

2. Нажмите на кнопку **New branch** для добавления новой ветви проекта (3.11).

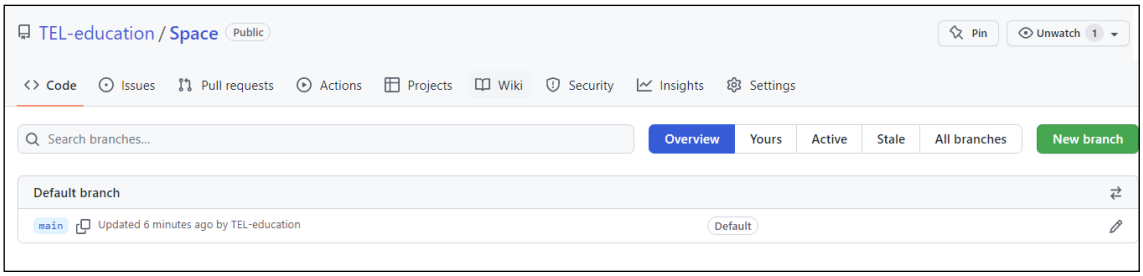


Рисунок 3.11 - Начало создания новой ветви в репозитории Space

3. Дайте новой ветви понятное и лаконичное название, например, `ВтораяВетвь`. Источником данных этой ветви будут данные ветви `main` – три файла проекта Space. Нажмите **Create branch** (рис.3.12). После создания новой ветви репозиторий включает две ветви (рис. 3.13). На данном шаге в каждой ветви сохранено по три одинаковых файла.

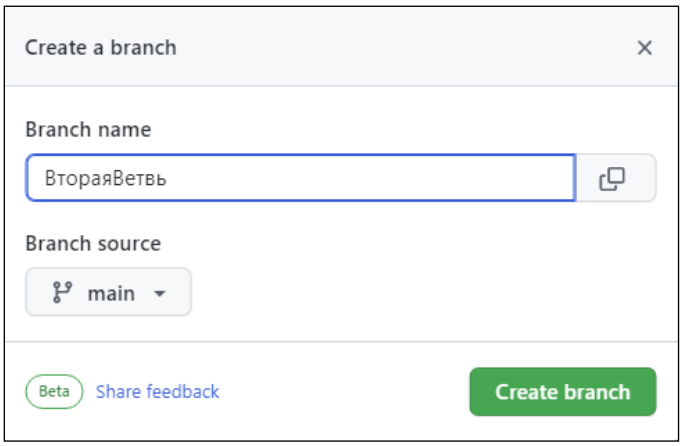


Рисунок 3.12 - Создание второй ветви в репозитории Space

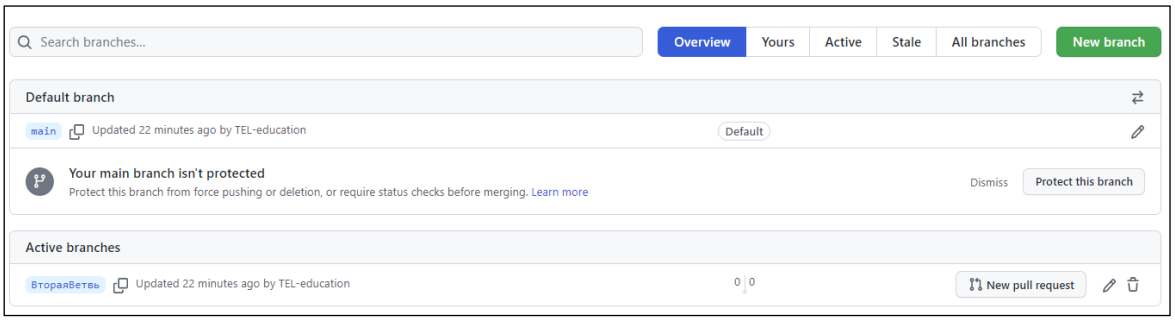


Рисунок 3.13 - Две ветви репозитория Space

При создании второй ветви GitHub потребует уточнений в политике безопасного слияния ветвей проекта. Нажмите **Protect this branche**. Будет произведен переход на новую страницу с политиками GitHub. В учебном проекте не активуйте чек-боксы, оставьте все настройки без изменений и примите соглашение о политике управления интеграцией.

4. Нажмите на название репозитория Space. Затем выберите из раскрывающегося меню название ВтораяВетвь (рис.3.14).

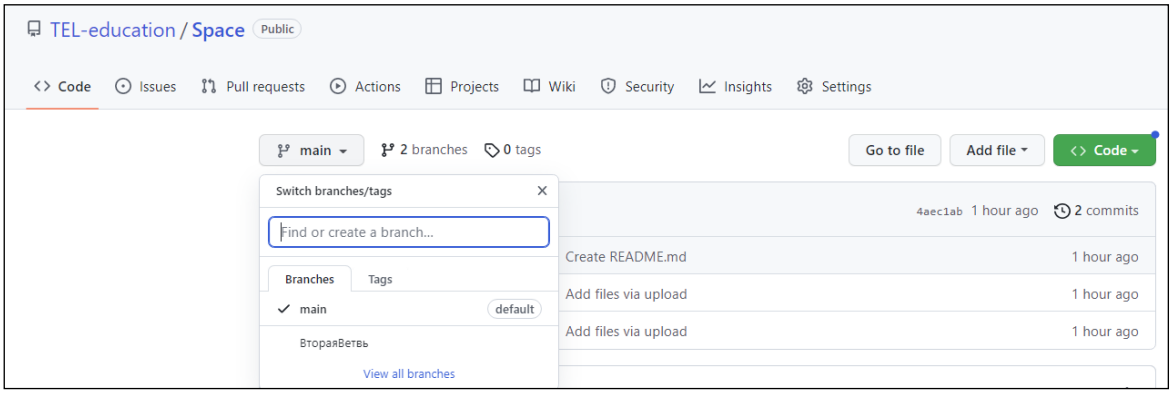


Рисунок 3.14 - Переход к данным, расположенным во второй ветви

5. Добавьте еще два файла во вторую ветвь репозитория. Для этого нажмите

на **Add files** и выберите из раскрывающегося списка **Upload files** (рис.3.15).

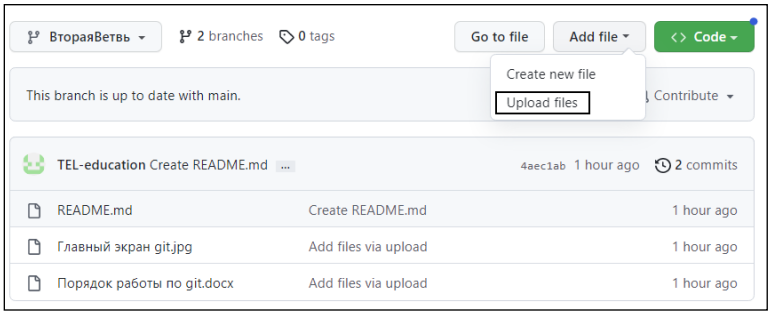


Рисунок 3.15 - Выбор варианта загрузки файлов

6. Скопируйте два файла из папки, подготовленной на персональном компьютере. Добавьте описание коммита и нажмите **Commit change** (рис. 3.16).

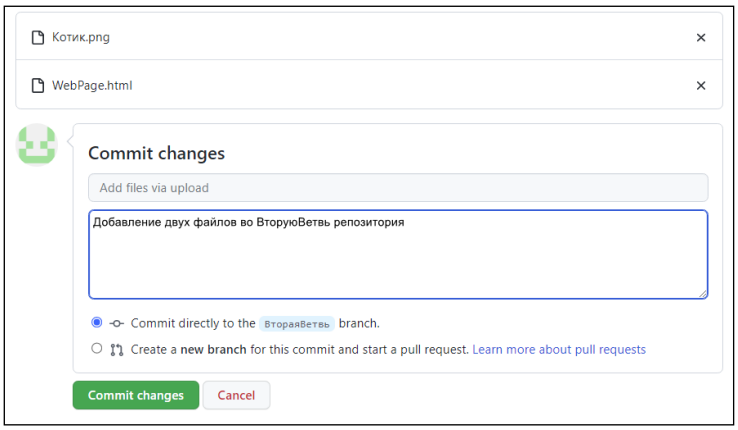


Рисунок 3.16- Добавление файлов во вторую ветвь

7. Нажмите на название репозитория Space и посмотрите на результат изменения количества файлов (рис.3.17).

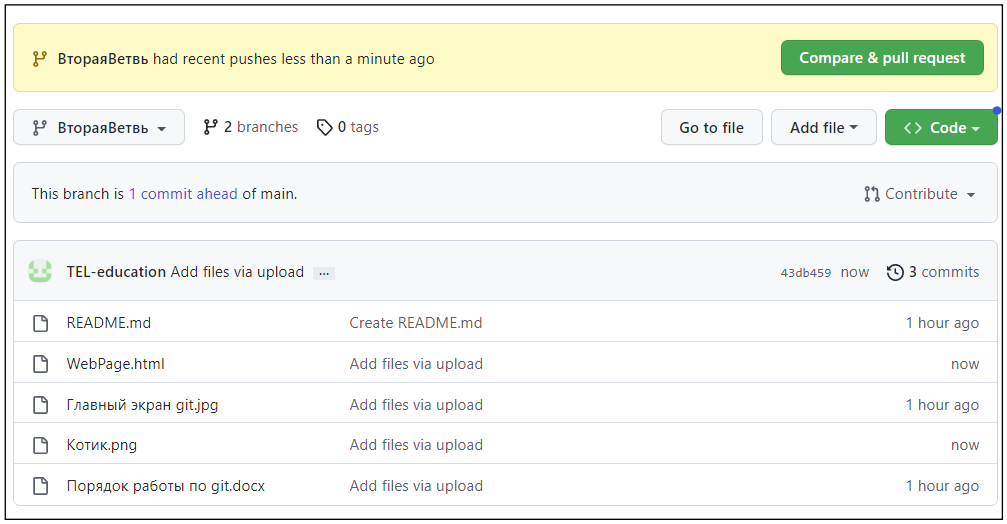


Рисунок 3.17 - Просмотр новых данных во второй ветви

Обратите внимание, что количество ветвей проекта увеличилось на единицу

и вместо `1 branches` записано `2 branches`.

Упражнение 4. Добавление третьей ветви проекта Space

1. Нажмите на пиктограмму `2 branches`, перейдите в область создания новой ветви (повторите шаги 2 и 3 предыдущего упражения). Назовите третью ветвь репозитория `ТретьяВетвь` (рис.3.18). Укажите, что источником данных для нее является также ветвь `main`.

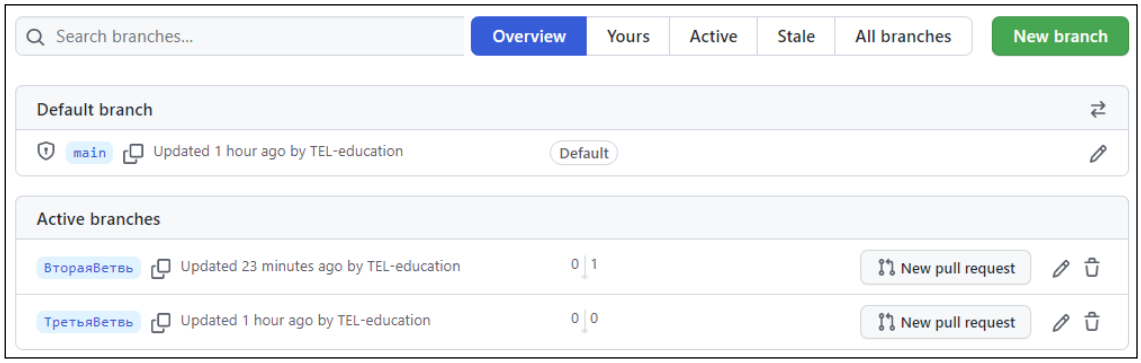


Рисунок 3.18 - Три ветви проекта

2. Перейдите в третью ветвь и создайте в ней текстовый файл. Для этого нажмите на **Add files** и в ракрывшемся списке выберите **Create New file** (рис. 3.17). Добавьте следующий текст: «GitHub – крупнейший веб-сервис для хостинга IT-проектов и их совместной разработки. Веб-сервис основан на системе контроля версий Git. Ознакомиться с документацией по работе с GitHub можно на сайте GitHub Help».

Назовите файл, например, `Назначение СКВ.txt`. Заполните комментарий и нажмите на кнопку **Commit new files** (рис.3.19).

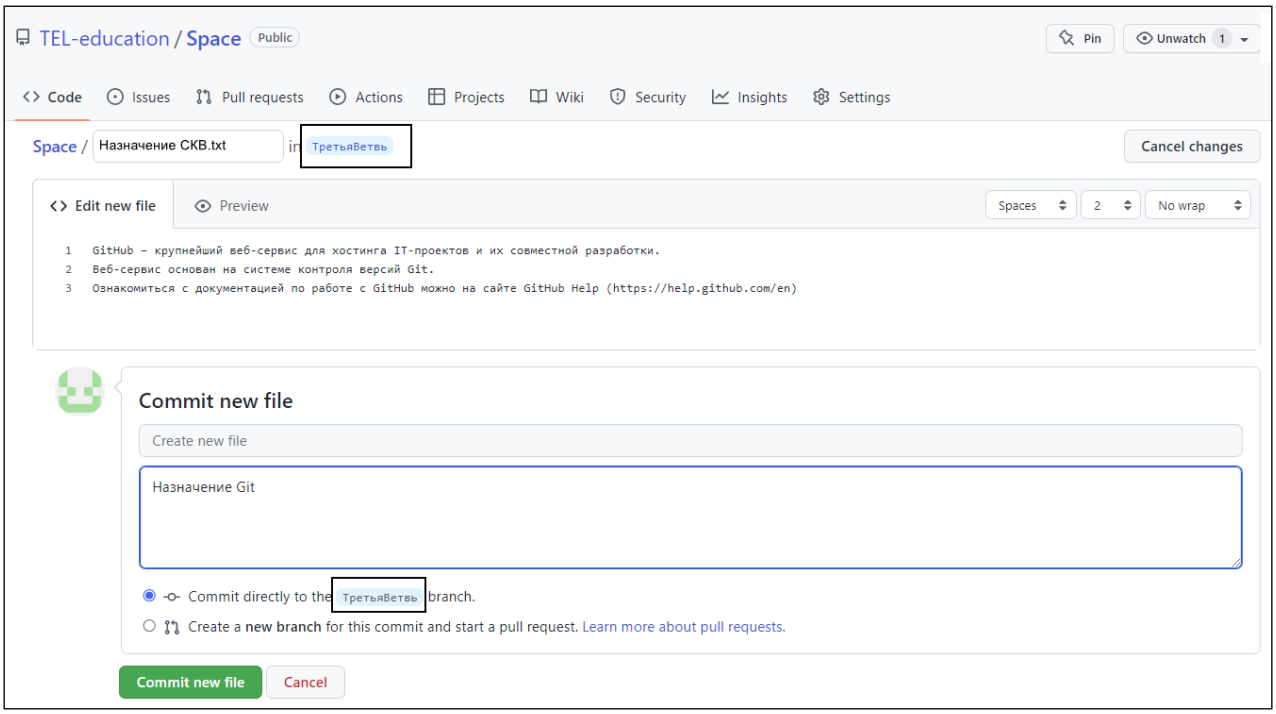


Рисунок 3.19 - Добавление текстового файла в третью ветвь репозитория

Обратите внимание, что добавление файла происходит в ветвь под названием ТретьяВетвь.

3. В репозитории созданы три ветви: `main`, `ВтораяВетвь`, `ТретьяВетвь` (рис.3.20).

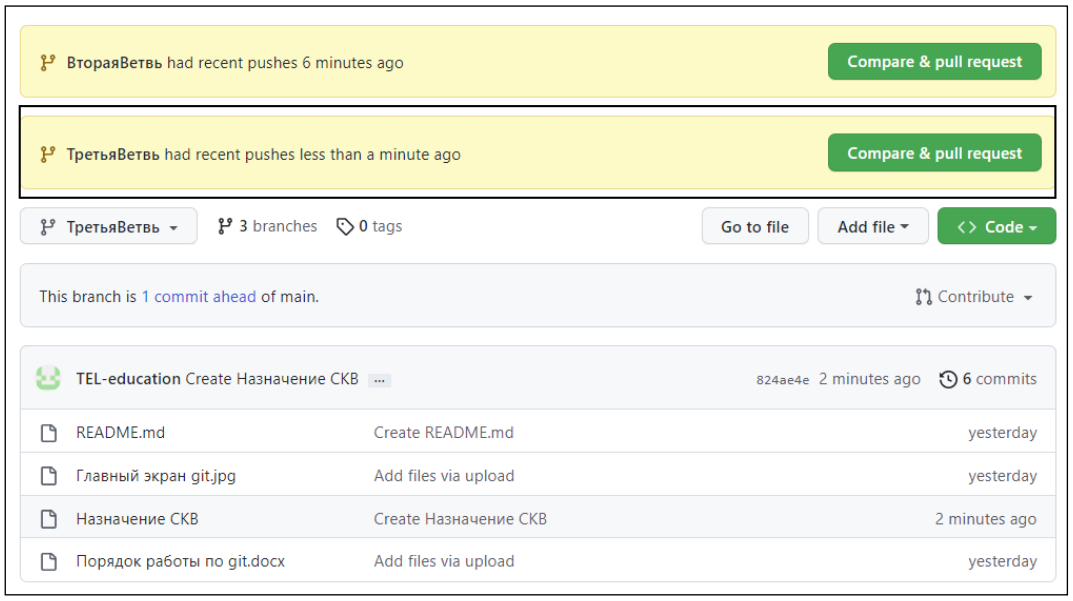


Рисунок 3.20 - Три ветви проекта

В ветви `main` сохранены исходные файлы проекта, в двух других – альтернативные варианты развития проекта, в которых к исходным файлам добавлены дополнительные. На следующем этапе разработчики могут выбрать, какой из

двух вариантов выбрать и, выполнив запрос на слияние, сделать его основным.

Упражнение 5. Объединение ветвей проекта Space

1. Проведите слияние третьей ветви проекта с основной ветвью. Нажмите на кнопку **Compare & pull request** (рис. 3.20). Pull request (пул-реквест) – это запрос на интеграцию изменений в выбранную ветвь проекта.

2. Проверьте правильность интеграционного потока: из `ТретьейВетви` в основную `main` (рис.3.20). Будет проведено автоматическое сравнение данных в двух ветвях и добавление в основную ветвь файлов, которые присутствуют только в `ТретьейВетви`.

3. Нажмите **Create Pull request** (рис.3.21).

4. Подтвердите запрос на интеграцию нажатием на кнопку **Merge Pull request** (рис.3.22).

5. Кнопка **Merge Pull request** изменила название на **Confirm merge** (рис.3.23).

6. Результат успешного завершения интеграции показан на рис.3.24.

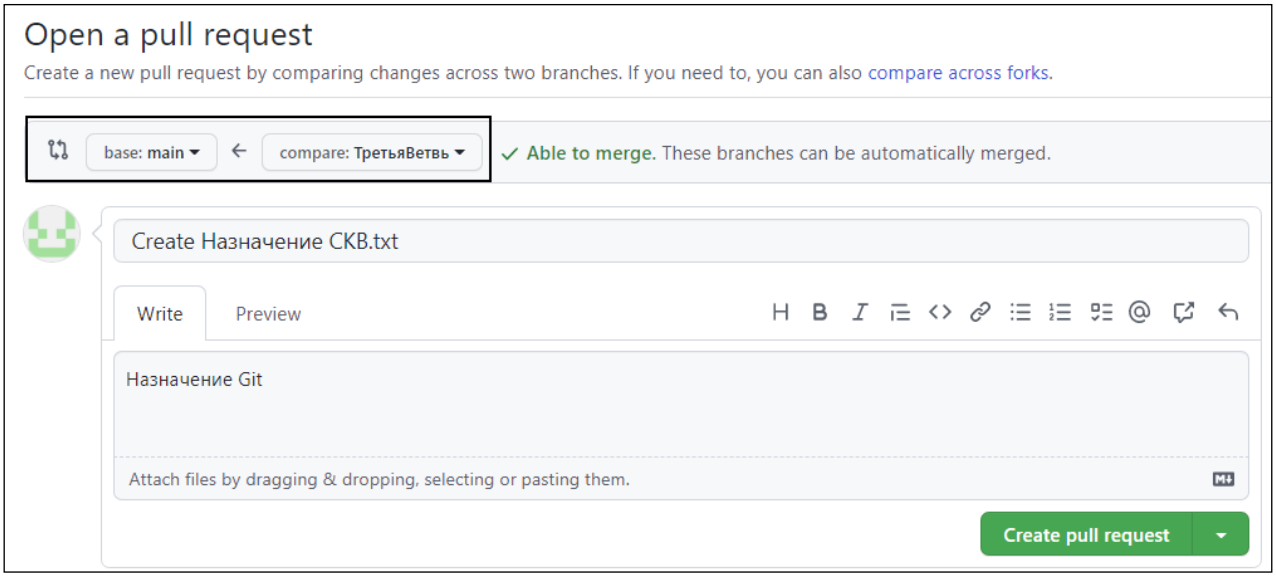


Рисунок 3.21 - Интеграция данных третьей ветви и основной

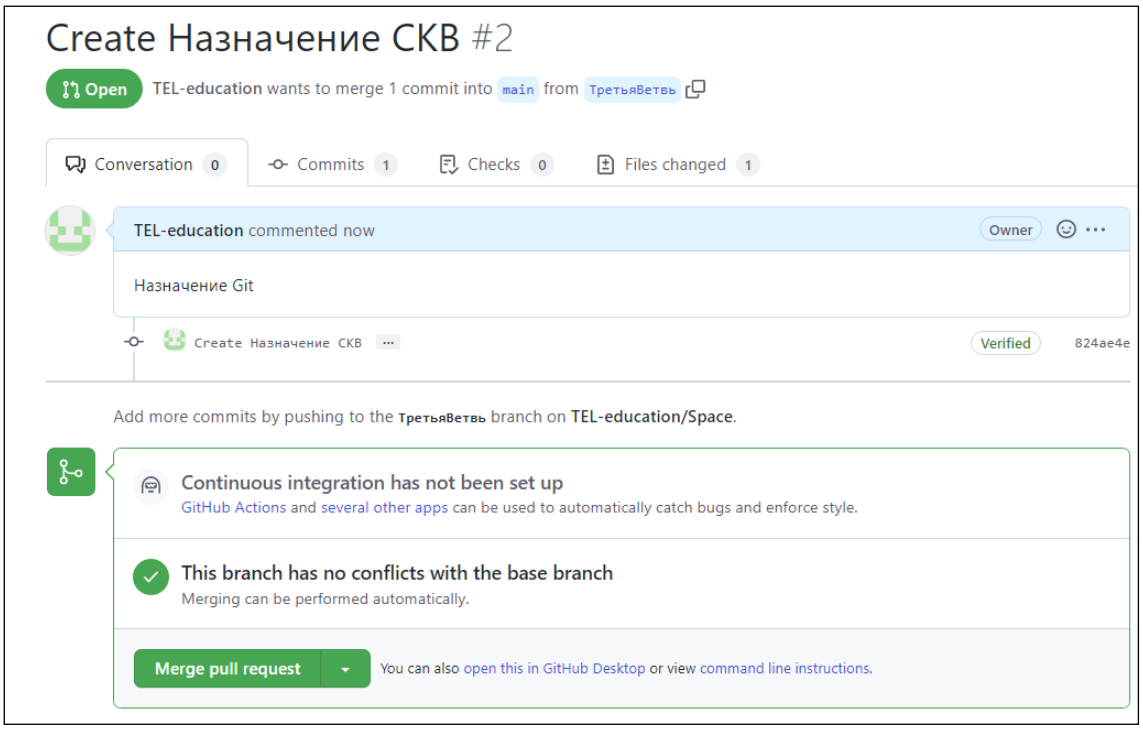


Рис. 3.22. Подтверждение запроса на интеграцию

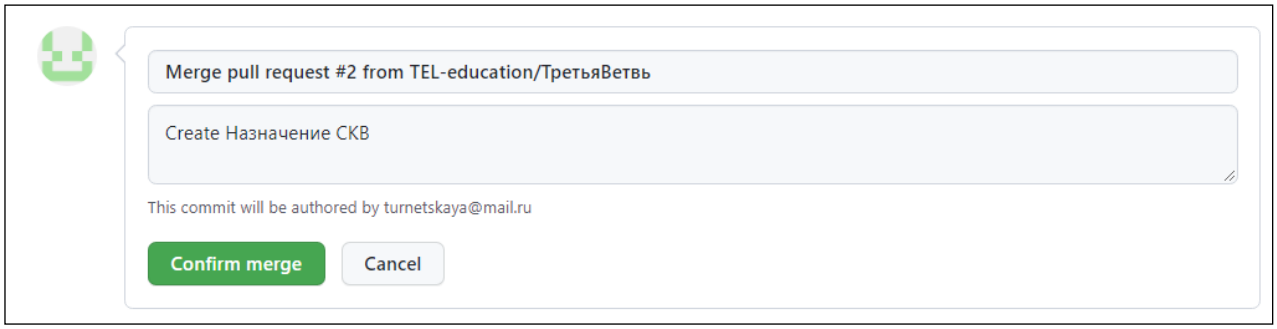


Рисунок 3.23 - Принятие подтверждения нажатием на кнопку Confirm merge

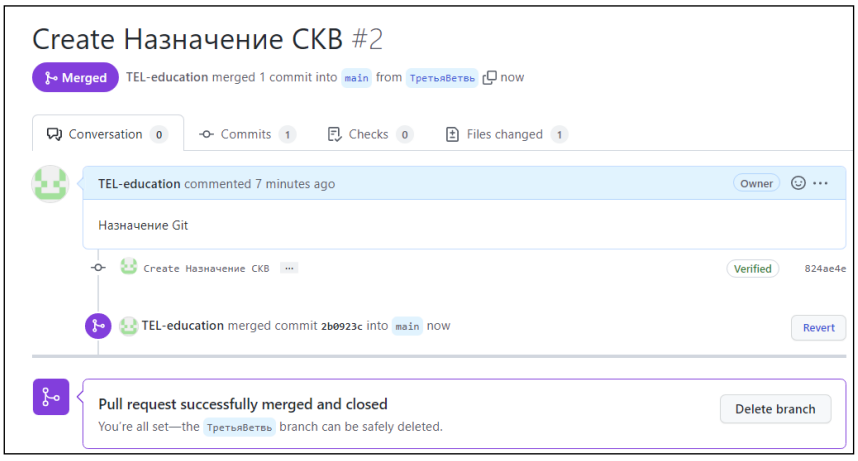


Рисунок 3.24 - Сообщение об успешной интеграции данных в основную ветвь

7. Перейдите в режим просмотра ветвей проекта: нажмите на **Space>3 branches**. В комментариях `ТретьейВетви` появилась информация о

выполненной интеграции (рис. 3.25).

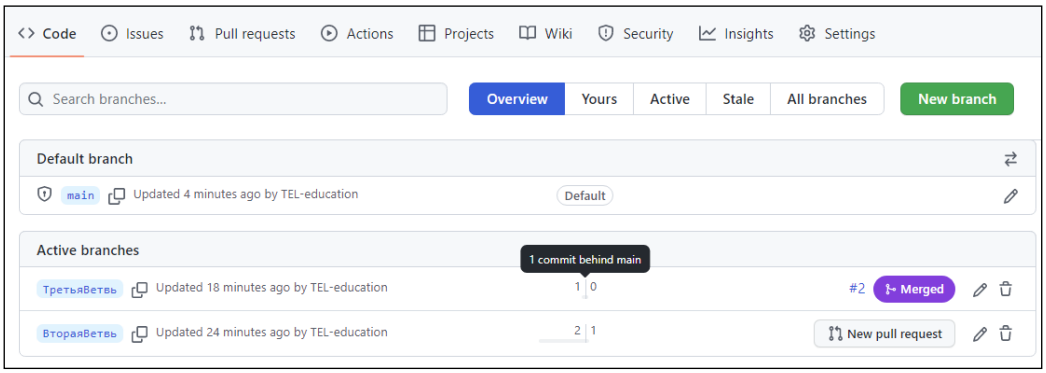


Рисунок 3.25 - Информация о действиях по ветвям репозитория

Упражнение 6. Форк репозитория

При работе над проектом используют множество библиотек, компонентов программного обеспечения или документацию, которая находится в открытом доступе и хранится на GitHub.

Для сохранения таких репозиториев в своем рабочем пространстве необходимо сделать их клонирование или форк. Для выполнения этой задачи.

1. Находят интересующий пользователя репозиторий, например https://github.com/electron/electron.

2. Находясь внутри выбранного для форка репозитория, нажимают Fork. Затем пошагово выполняют действия по клонированию (рис.3.26). На первом шаге предложено создать репозиторий с таким же именем, как оригинальный репозиторий (тот, который требуется форкнуть) (рис.3.27).

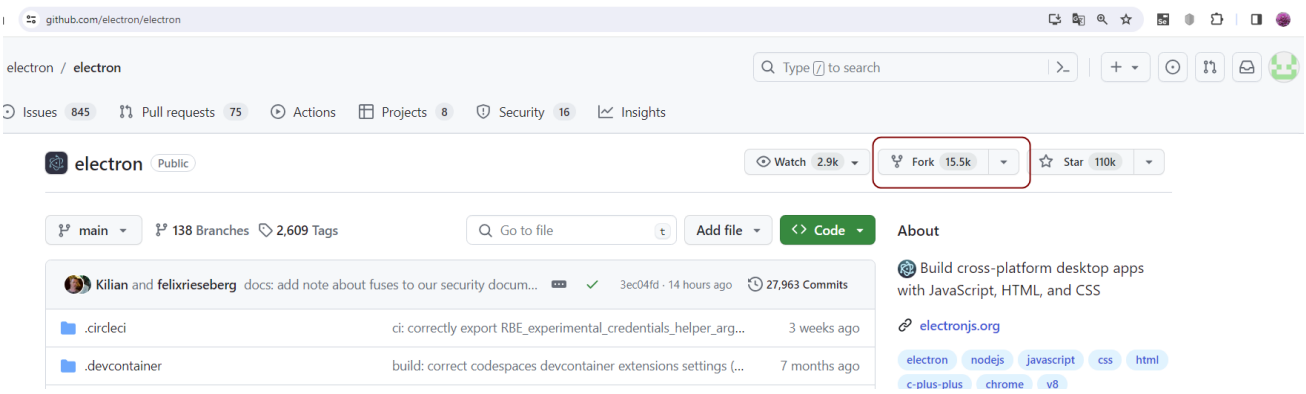


Рисунок 3.26 - Нажатие на Fork

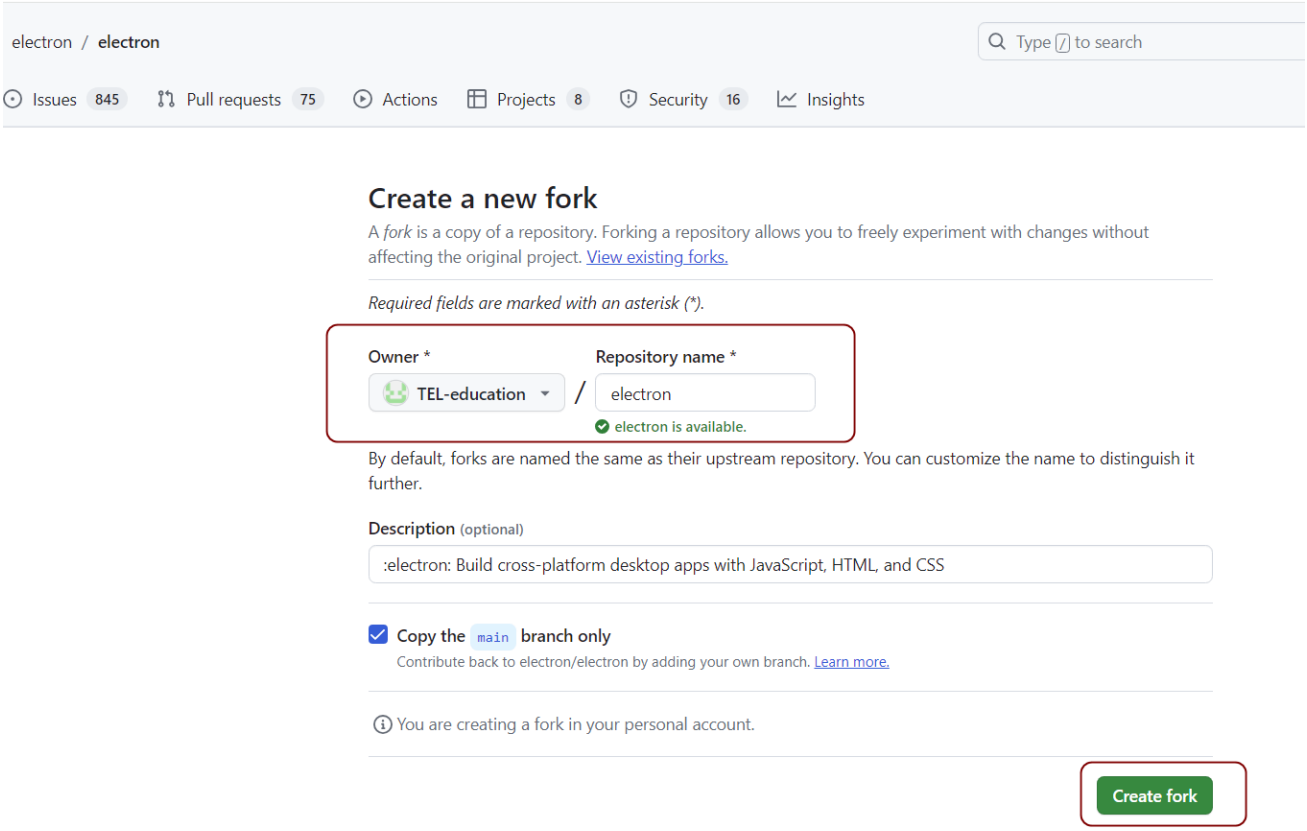


Рисунок 3.27 - Создание нового репозитория в рабочем пространстве

Автоматически будет создан репозиторий и осуществлен перенос (рис. 3.28). Обратите внимание, что в результате выполненных шагов, появляется новый репозиторий в вашей рабочей области.

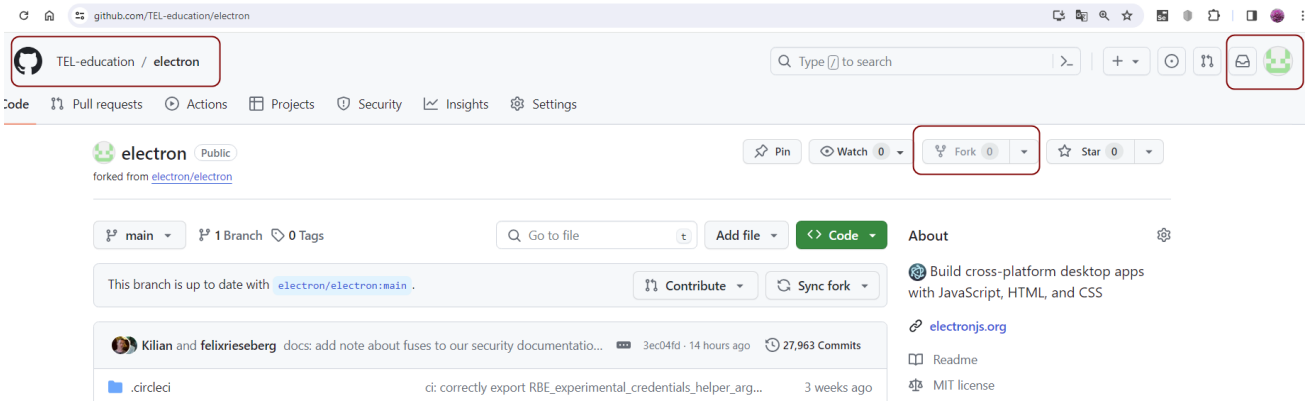


Рисунок 3.28 - Форк репозитория в рабочем пространстве пользователя

5. ВЫПОЛНЕНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Индивидуальное задание рекомендуется выполнять в IDE VS Code, так как вы использовали его в лабораторной работе №4.

1. Для работы необходимо установить систему управления версиями git. Установочный файл доступен на официальном сайте проекта https://git-scm.com/install/windows (можно выбрать другую операционную систему). Также в редакторе VS Code необходимо установить расширение GitHub Pull Requests, предназначенное для работы с репозиториями GitHub и Pull Request непосредственно из среды разработки. После установки расширения необходимо выполнить авторизацию в GitHub с использованием своей учетной записи.

2. Вы можете сначала создать репозиторий на сайте GitHub, а затем связать свою рабочую папку проекта с ним. Либо можно автоматически создать репозиторий через VS Code. В этой работе необходимо использовать рабочую папку из **прошлой лабораторной работы (ЛР№4)** с созданным лендингом. Для начала инициализируем репозиторий (рис. 4.1).

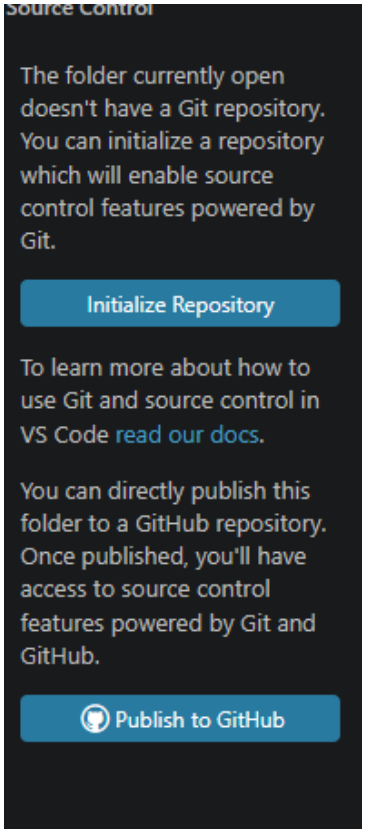


Рисунок 4.1 - Инициализация репозитория

На вкладке Source Control в VS Code появятся ваши файлы (рис. 4.2).

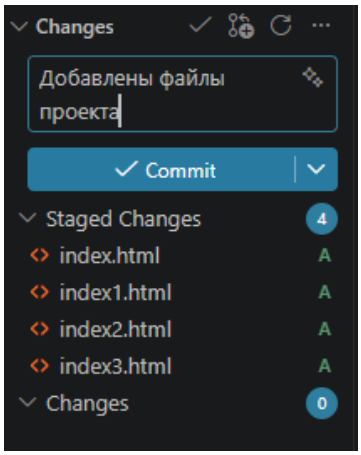


Рисунок 4.2 – Файлы в рабочей папке

Перед тем как нажимать **Commit** необходимо ввести комментарий – название изменения. После нажатия **Commit**, если вы выполняете это первый раз, потребуется ввести *почту и имя пользователя*. Не забудьте это сделать, иначе ничего не получится.

Если вы уже создали репозиторий в Github, в который хотите добавить проект, то нужно выбрать **Remote – Add Remote** и выбрать GitHub, а затем свой уже созданный репозиторий (рис. 4.3).

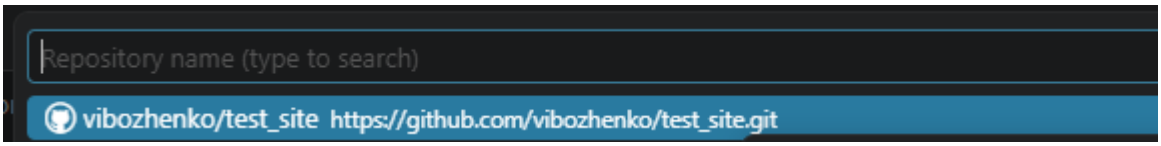


Рисунок 4.3 – Выбор репозитория

После необходимо нажать кнопку Publish branch, чтобы изменения отобразились на GitHub (в пустую папку добавятся ваши файлы с сайтом).

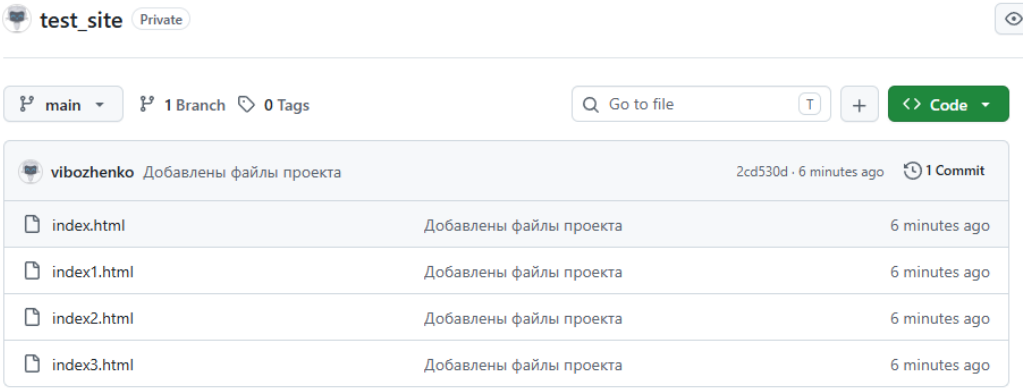


Рисунок 4.4 – Репозиторий на GitHub

Для примера попробуем изменить название и отобразить изменения на GitHub (рис.4.5-4.6). Видно, что в редакторе кода отображается старый заголовок

и новый (то есть мы можем отследить изменения). Не забудьте указать название изменений перед нажатием кнопки Commit.

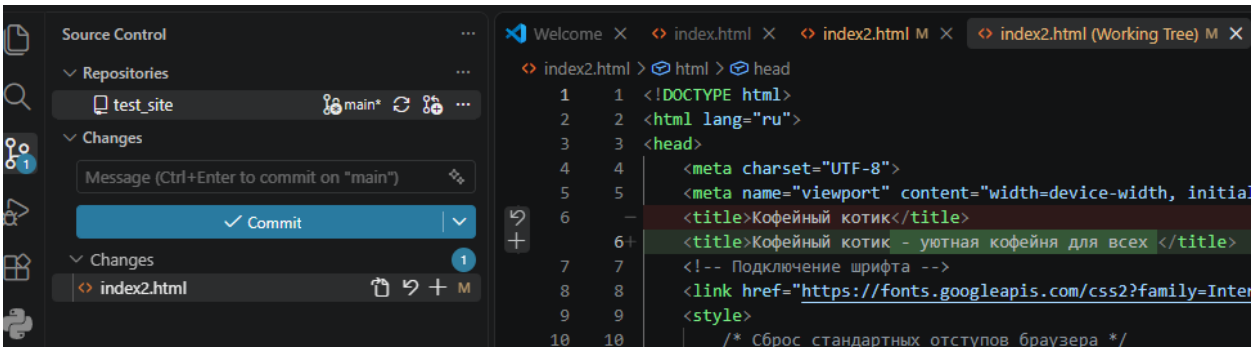


Рисунок 4.5 – Изменение названия сайта

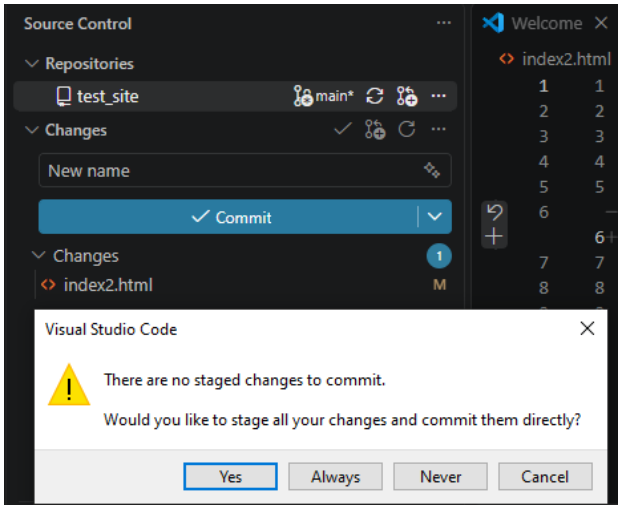


Рисунок 4.6 – Выполнение Commit

Важно помнить, что commit сохраняет изменения **локально**, то есть после нажатия Commit в GitHub ничего не появится. Нужно нажать Sync – чтобы сделать push изменений – отправить код на github. Видно, что теперь для файла index2.html другой комментарий «New name», потому что мы внесли изменения именно там (рис. 4.7.).

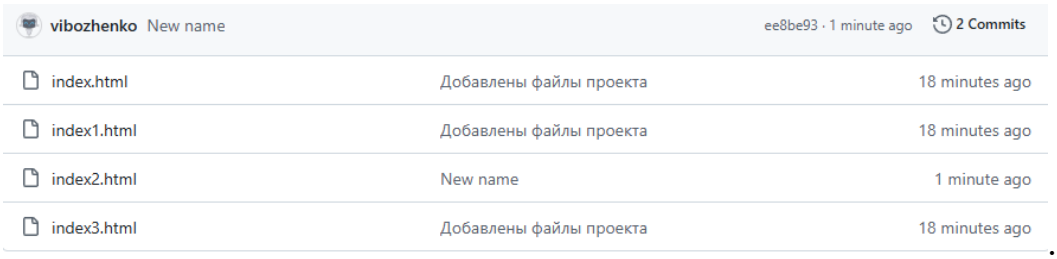


Рисунок 4.7 – Github после внесения изменения

Если у вас уже создан репозиторий на GitHub с другим проектом (не пустой), и вы хотите с ним работать в VS Code, то можно выполнить команду clone и склонировать репозиторий себе в VS для работы.

3. Следующий шаг – дальнейшая **модернизация сайта**. В прошлой работе мы исправили только некоторые недочеты интерфейса. При этом у вас должен быть **список того, что ещё необходимо сделать из предыдущей работы**. Либо можно опять попросить LLM или агента проверить сайт, чтобы понять, что можно изменить.

Опишите, что будете исправлять в своей предметной области (предоставьте скриншоты до и после изменения).

Например, на сайте кофейни в результате работы агента произошло наложение картинки и эмоджи (рис. 4.8). Можно попросить агента исправить конкретно этот один недочёт, а после изменения отправить их на GitHub с комментарием, что было сделано.

В индивидуальном задании вам необходимо будет выполнить как минимум один такой commit в основную ветку проекта.



Рисунок 4.8 – Проблема с изображением на сайте

Следующая проблема на сайте – это отображение фото в меню (рис. 4.9), некоторые фото не отображаются.

В индивидуальном проекте вам нужно также использовать какие-то фотографии. Если у вас на данном этапе вообще нет изображений на сайте – подумайте, в каком блоке можно их добавить. Выполните добавление изображений это с помощью нейросети.

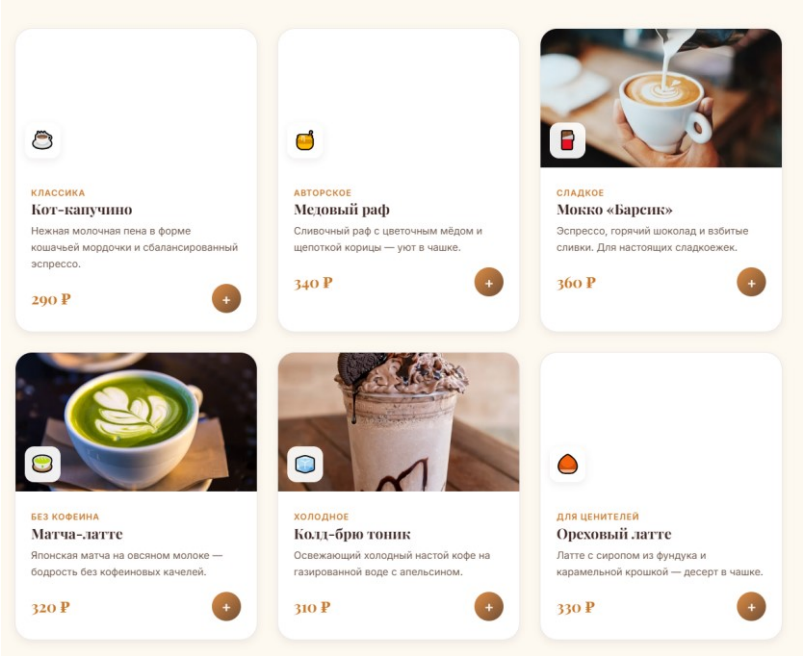


Рисунок 4.9 – Проблема с изображениями в меню

Рекомендуется найти изображения с открытой бесплатной лицензией (*можно использовать специальные сайты, например RuPixel*). После этого необходимо добавить их в рабочую папку проекта (рис.4.10.). Обратите внимание, что у нас в проекте произошло разделение на три файла из одного index.html – структура, style.css – оформление, main.js – логика. Стоит понимать, что при смешивании HTML и CSS в одном файле нарушаются принципы SOLID (single responsibility – один файл отвечает сразу и за структуру, и за оформление), DRY (стили дублируются между страницами), KISS (частично код становится сложнее). Агент в ходе своей работы исправляет это и следуют принципам чистого кода.

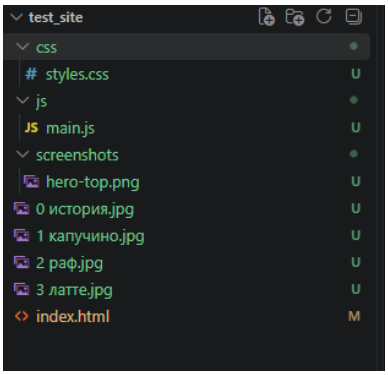


Рисунок 4.10 – Добавление изображений в папку проекта

Затем необходимо написать агенту промт (*сформулируйте его сами или с помощью нейросети, обязательно укажите, как был создан промт*). Например, для кофейни:

«*Измени фото: вместо интерьер уютной кофейни используй 0 история. вместо классический капучино в белой чашке - 1 капучино, вместо латте с рисунком корицей - 2 раф, вместо флэт уйат в керамической чашке - используй изображение 3 латте.* ***Внеси изменения на сайт, вставь картинки из рабочей папки проекта***».

Обратите внимание, что мы также могли изменить фото, которые уже отображались корректно. Вам необходимо составить **свой промт** и поменять или добавить минимум **3 фотографии на вашем сайте**.

Агент после выполнения вашего задания будет работать поэтапно (рис. 4.11). Результат на рис.4.12. Обязательно приводите скриншоты работы агента.

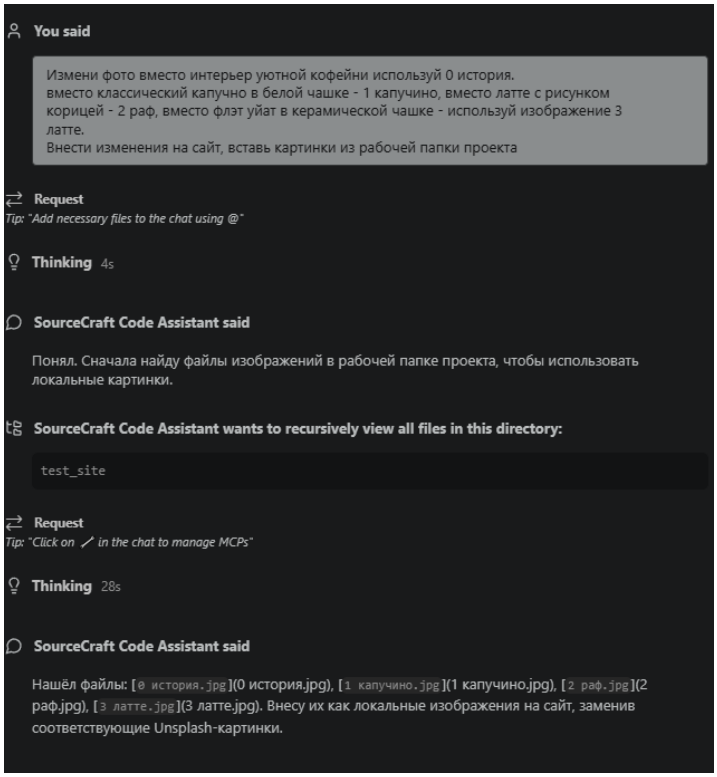


Рисунок 4.11 – Работа агента для добавления новых фото в проект (фрагмент)

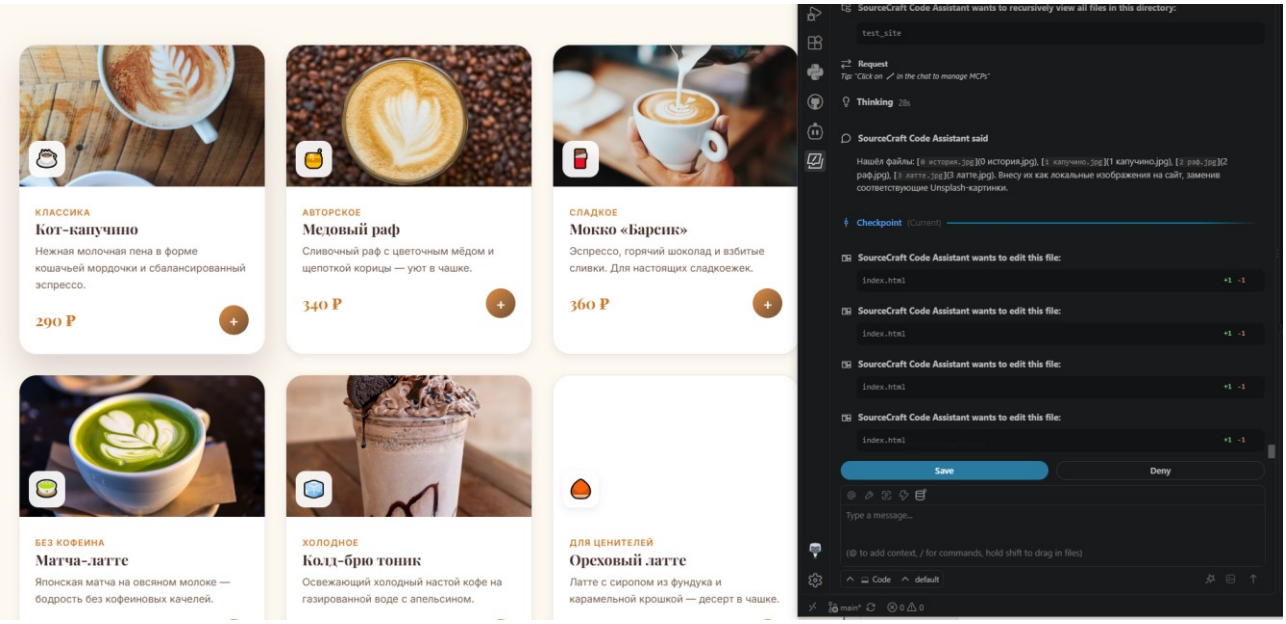


Рисунок 4.12 – Результат добавления фото

Посмотрим, как теперь выглядит блок Source Control в VS Code (рис.4.13). Видно, что мы также удалили другие версии сайта и оставили только один дизайн в index.html.

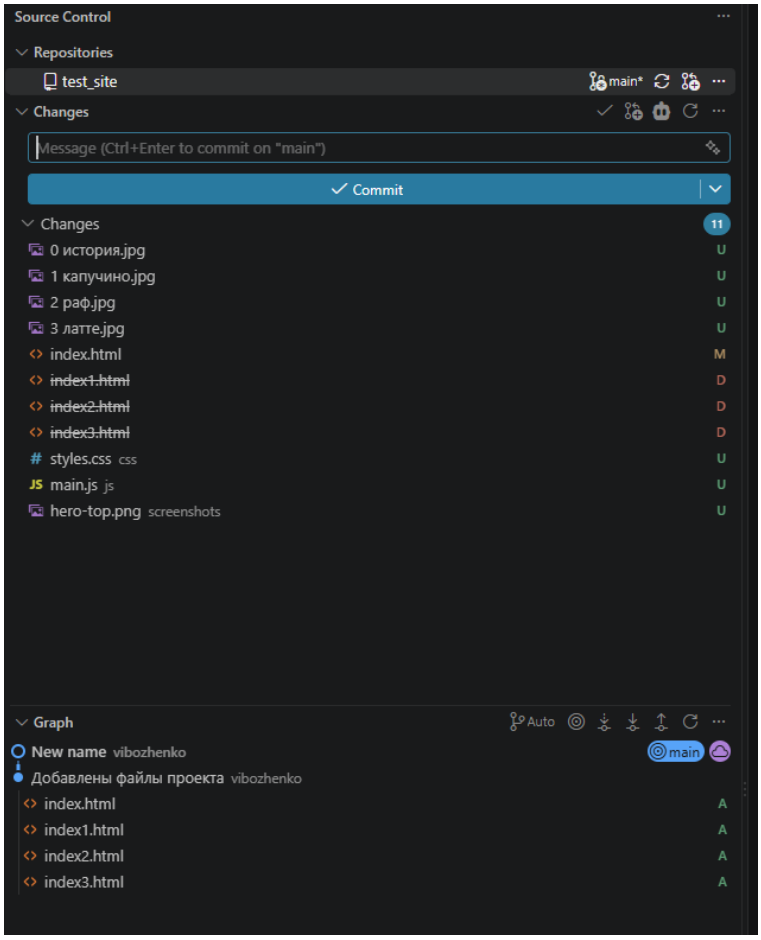


Рисунок 4.13 – Вкладка Source Control после добавления фото

При нажатии commit изменения сохранятся локально, для отправки на GitHub не забудьте сделать push. Результат на GitHib на рис 4.14.

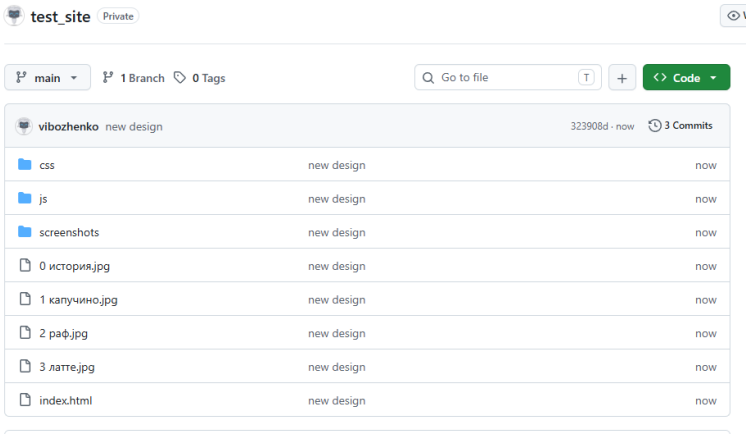


Рисунок 4.14 – Новый дизайн на Github

Обратите внимание, что в случае такого Commit у вас остаётся одна ветка, а старые файлы перестают быть доступны. То есть вы не сможете вернуться к другой версии проекта.

Поэтому в индивидуальной работе используйте **ветки** (осуществите добавление или изменение фото и **сделайте это именно в новой ветке**).

Рассмотрим пример, как можно создать ветку в VS Code. Например, давайте удалим эмоджи из меню, попросим агента внести изменения в код, а затем создадим новую ветку в VS Code (рис. 4.15).

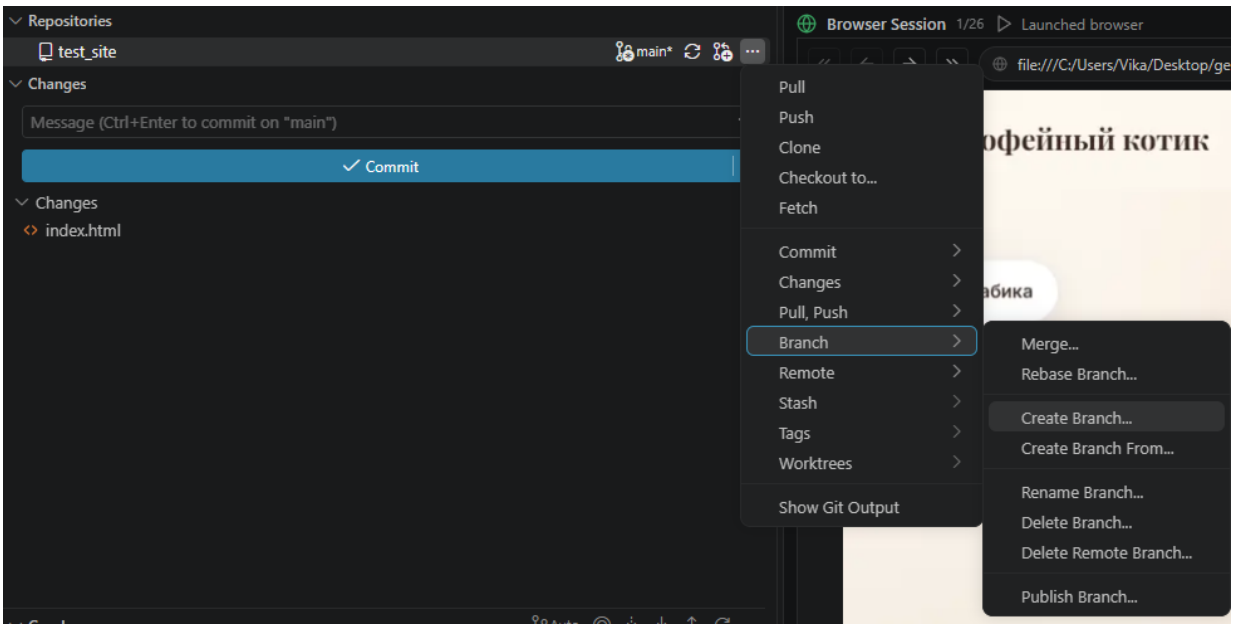


Рисунок 4.15 – Создание новой ветки со своим названием

Видно, что теперь вместо main отображается delete_emoji в VS Code (рис. 4.16). Важно отслеживать, в какой ветке вы работаете, и какие изменения отправляете на GitHub. Перед нажатием Commit не забудьте ввести название (рис.4.17).

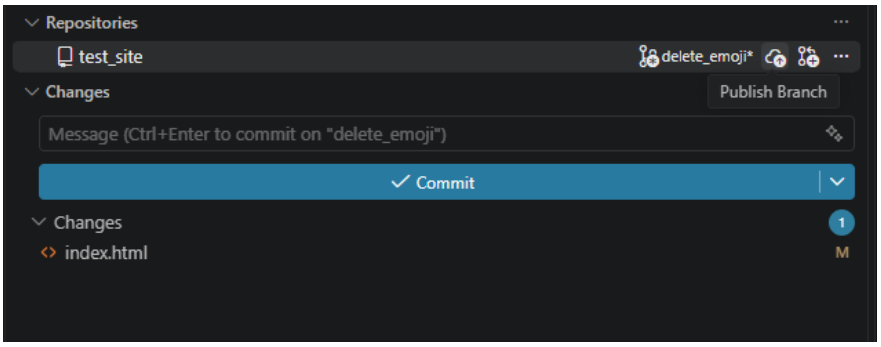


Рисунок 4.16 – Новая ветка

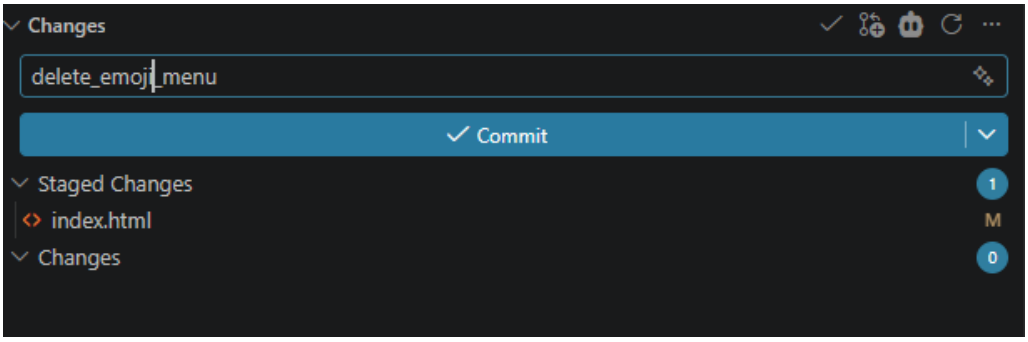


Рисунок 4.17 – Ввод названия

Результат изменения сайта на рис. 4.18. В вашем GitHub также отображается несколько веток (рис. 4.19).

Теперь можно решить, как лучше выглядит сайт (*с эмоджи или без них*) и **сделать какую-то из веток основной**.

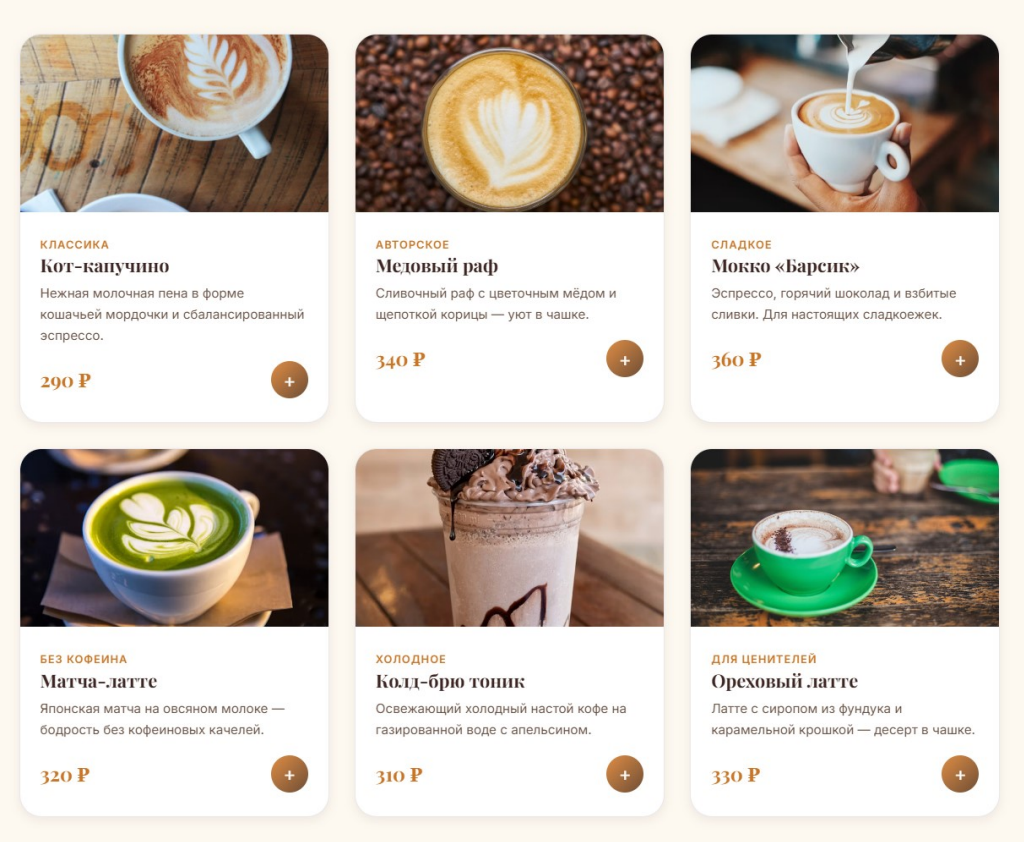


Рисунок 4.18 – Измененное меню (без эмоджи)

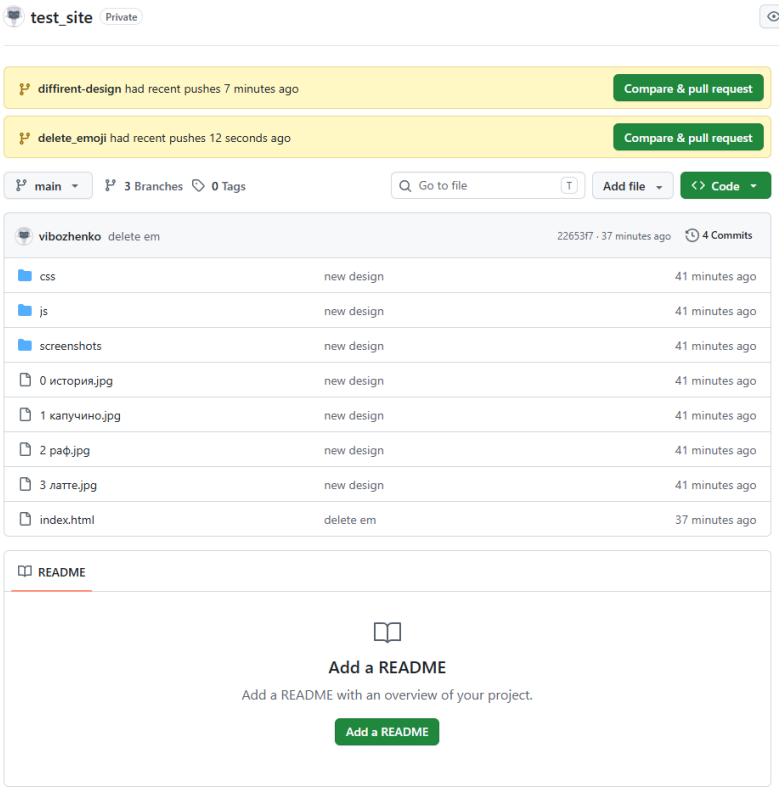


Рисунок 4.19 – Репозиторий в GitHub с несколькими ветками

Объединим ветки, всё-таки без эмоджи сайт смотрится лучше, поэтому объединим версию (ветку) **main** с веткой **delete_emoji**. Можно сделать это через VS Code. Например, нажимаем Ctrl+Shift+P и вводим Git: Merge



и выбираем ветку, с которой хотим объединить main из выпадающего списка (рис. 4.20).

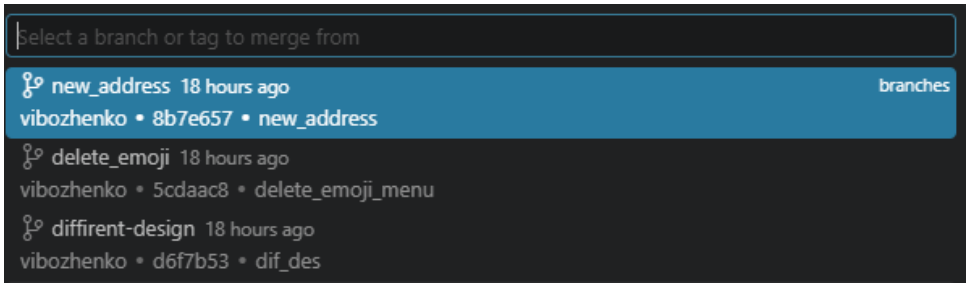


Рисунок 4.20 – Объединение веток

В вашей индивидуальной работе вам также нужно будет **показать создание нескольких веток** сайта с разными изменениями, а также **объединение веток**.

После того, как вы внесли все необходимые изменения, интерфейс сайта вас устраивает – выполним публикацию с помощью GitHub.

Переходим в настройки репозитория (settings), открываем Pages, выбираем ветку Branch=main (рис. 4.21).

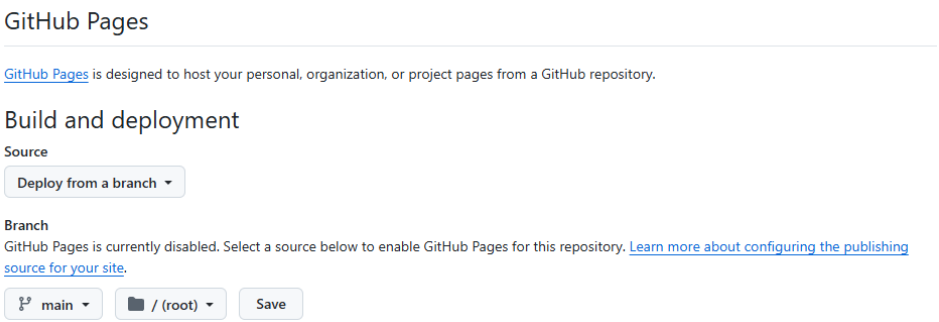


Рисунок 4.21 – Создание pages

В результате в репозитории проекта на GitHub появится Deployments (рис.4.22), и вам будет доступна ссылка вида https://USERNAME.github.io/REPO. https://vibozhenko.github.io/test_site/ – вот пример сайта с кофейней.

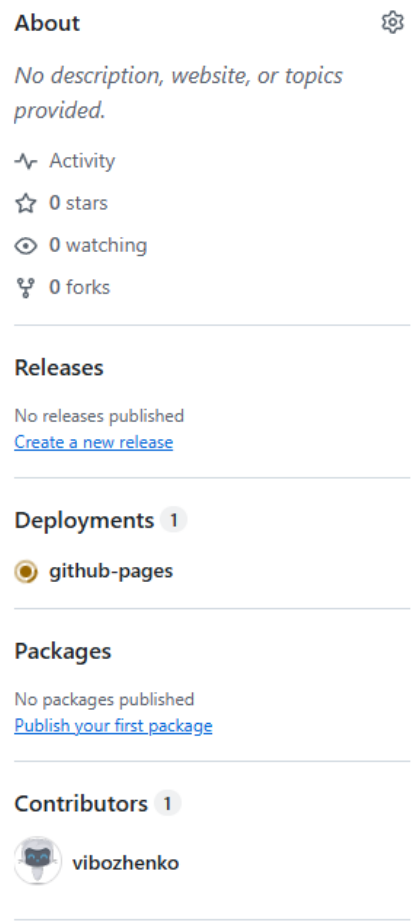


Рисунок 4.22 – Создание pages

Вам также необходимо приложить **ссылку на ваш опубликованный сайт** в GitHub. Обратите внимание, что репозиторий должен быть публичный.

Кроме того, вы можете внести изменения в проект уже после публикации, тогда у вас будет несколько версий проекта (рис. 4.23). Возможно, преподаватель после защиты работы попросит вас выполнить какие-то изменения, которые вы сможете внести, и они сразу отразятся на вашем сайте.

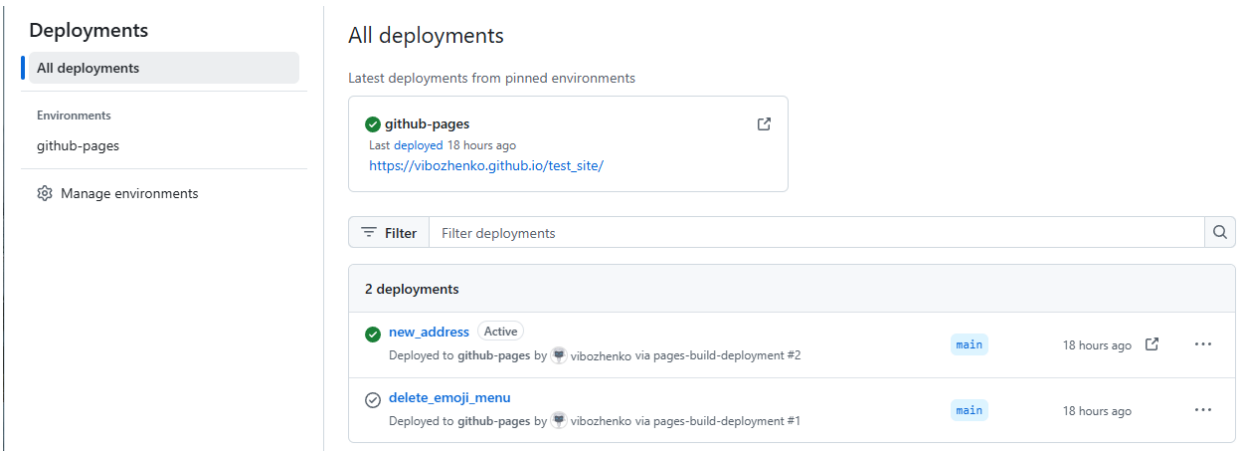


Рисунок 4.23 – Вкладка Deployments

6. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. 100 самых ценных репозиториев GitHub URL: https://habr.com/ru/articles/453444/(дата посещения 03.02.2026)

2. Программная инженерия. Интеграционный подход к разработке / Е. Л. Турнецкая, А. В. Аграновский. — Санкт-Петербург: Лань, 2023. – 216 с. – ISBN 978-5-507-46898-0. – Текст : электронный // Лань: электронно-библиотечная система. – (access : https://e.lanbook.com/book/352307).

3. Отчет GitHub за 2023 г. URL: https://github.blog/2023-11-08-the-state-of-open-source-and-ai/ (дата посещения 03.02.2026)

ПРИЛОЖЕНИЕ А

ТИТУЛЬНЫЙ ЛИСТ ОТЧЕТА

Бланк титульного листа отчета по лабораторной работе скачайте по ссылке https://guap.ru/c/regdocs/docs/uch

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА №41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

_____	_____	_____
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ЛАБОРАТОРНАЯ РАБОТА №5

«ОРГАНИЗАЦИЯ РЕПОЗИТОРИЯ В СИСТЕМЕ УПРАВЛЕНИЯ ВЕРСИЯМИ GIT»

по дисциплине: ИНФОРМАТИКА

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № _____	_____	_____
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

ПРИЛОЖЕНИЕ Б

ФРАГМЕНТ ПРИМЕРА ОТЧЕТА

Цель и задачи практической работы

Цель: получение практических навыков работы с распределенной системой управления версиями Git и сервисом GitHub.

Для достижения поставленной цели требуется решить следующие задачи:

1. Создать пространство проекта на сервисе GitHub.

2. Разработать три ветви проекта.

3. Выполнить форк внешнего репозитория.

Выполнение практической работы

1. Создание аккаунта на GitHub

На первом этапе выполнения практической работы необходимо создать аккаунт на GitHub. Ниже на рисунке 1 представлен скриншот личного аккаунта.

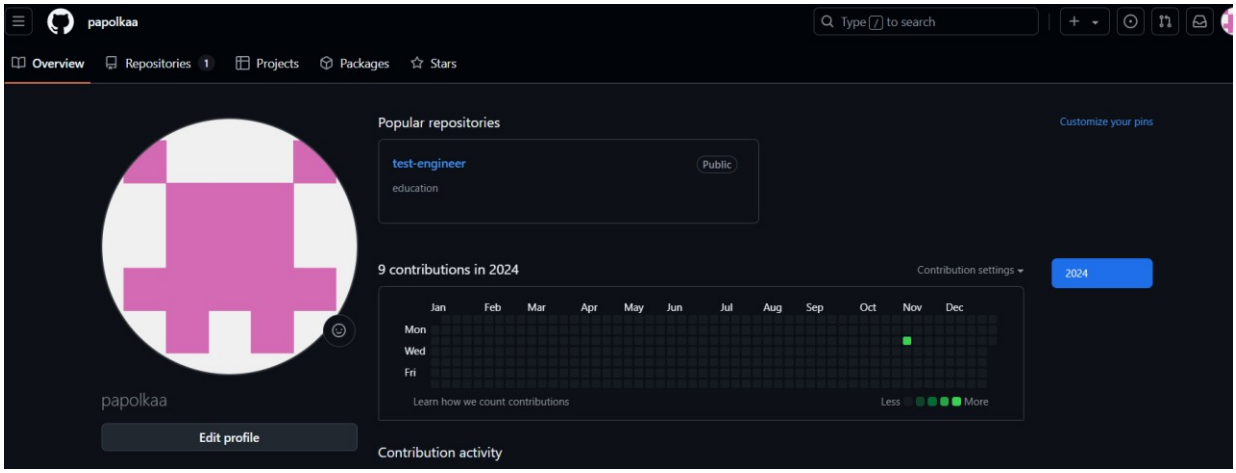


Рисунок 1 – Аккаунт на GitHub

2. Создание трёх ветвей репозитория проекта «test-engineer»

Далее было необходимо создать три ветви проекта (рисунок 2).

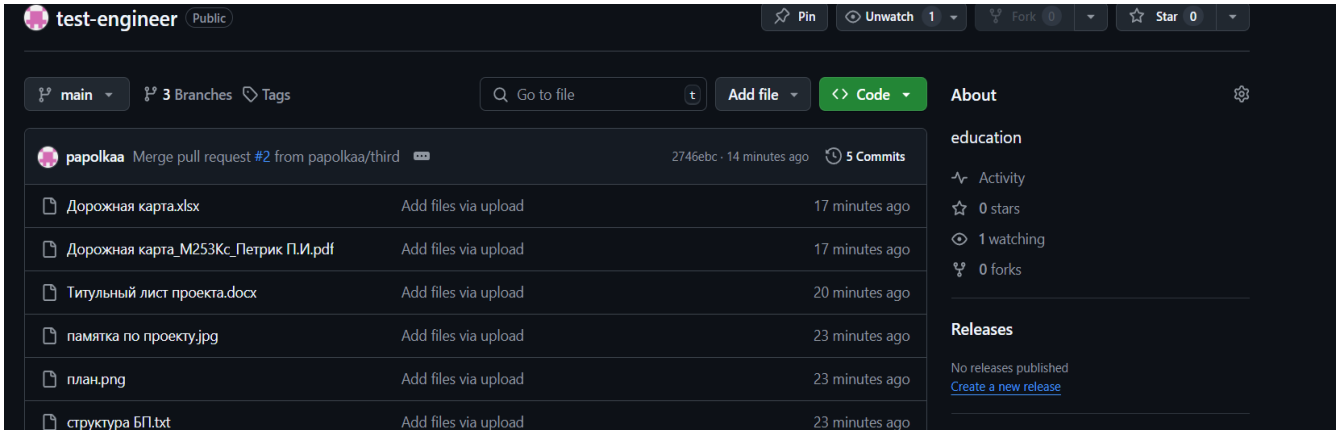


Рисунок 2 – Ветвь main

Для первой ветви были выбраны три файла разных форматов – txt, png, jpg, для второй ветви один файл – docx, для третьей ветви выбраны два файла – pdf, xlsx. Ниже на рисунке 3 представлены три ветви проекта.

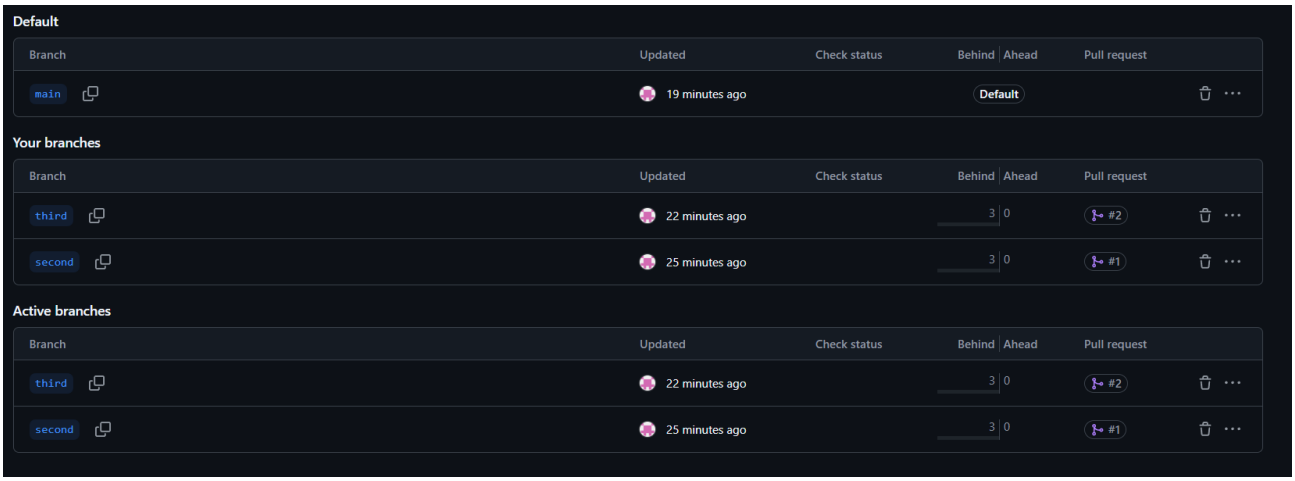


Рисунок 3 – Ветви проекта

3. Копия проекта с полезными библиотеками

Ниже на рисунке 4 представлен скриншот «nuttx» проекта с полезными библиотеками.

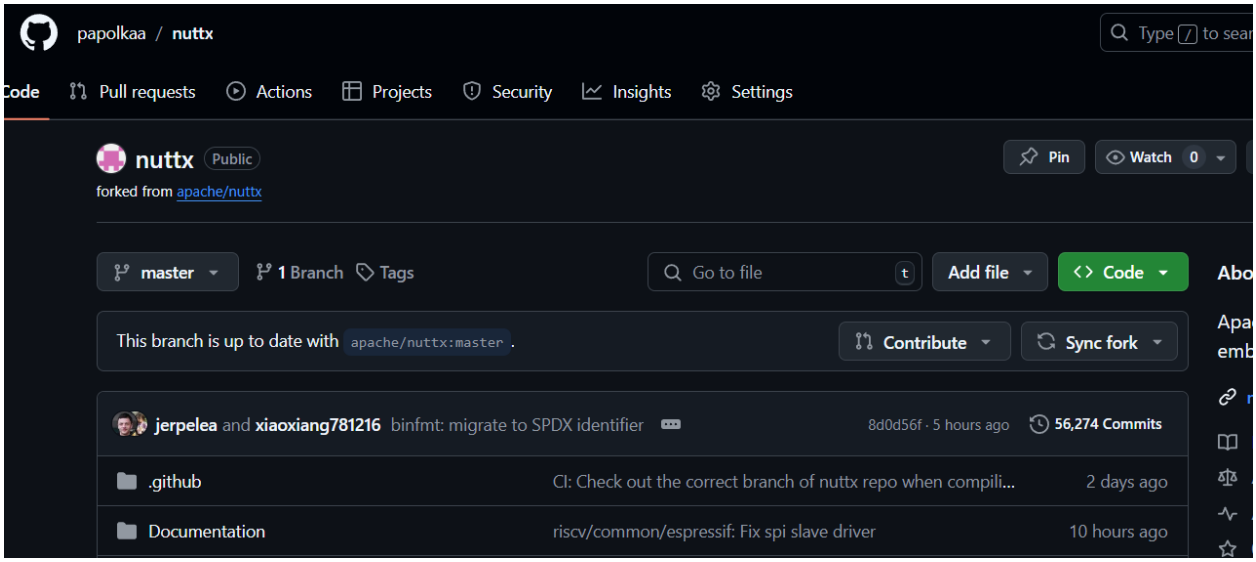


Рисунок 4 – Копия библиотеки «nuttx»

4. Ссылка на удаленный репозиторий на GitHub

Далее необходимо дополнить лабораторную работу индивидуальным заданием и скриншотами к нему по аналогии с методическими рекомендациями

Вывод

Целью данной практической работы было освоение базовых навыков работы с системой контроля версий Git и сервисом GitHub, что имеет ключевое значение для эффективного ведения проектов в области разработки программного обеспечения. В рамках работы предстояло создать проект на GitHub, развить навыки управления ветвями, фиксации изменений, а также изучить процесс форка и клонирования внешних репозиториев. Эти знания являются основой для командной разработки, где отслеживание изменений, сохранение истории версий и возможность отката к предыдущим состояниям играют важную роль.

В процессе работы были освоены такие важные навыки, как регистрация и создание репозиториев на GitHub, создание и управление ветвями, фиксирование изменений в каждой из ветвей, а также работа с различными форматами файлов. Особое внимание уделялось управлению ветвями, что позволило понять, как можно работать над разными аспектами проекта независимо друг от друга. Практика в фиксировании изменений и комментарии к коммитам помогли лучше

понять важность логичного и понятного описания каждого изменения для командной работы.

Одной из возникших сложностей стало освоение работы с несколькими ветвями, особенно при их слиянии. Конфликты при объединении ветвей стали неожиданным препятствием, поскольку требовали понимания основ разрешения конфликтов и дополнительного времени на корректировку изменений. Для решения этих проблем были изучены команды Git, предназначенные для просмотра и разрешения конфликтов, что позволило успешно завершить слияние ветвей и зафиксировать результаты. Дополнительно, сложность возникала и при работе с удаленными репозиториями, например, при форке стороннего проекта. Здесь требовалось разобраться в процедуре клонирования и последующего обновления репозитория, чтобы интегрировать новые изменения.

В итоге выполнение данной работы дало не только технические знания, но и опыт преодоления практических трудностей, с которыми сталкиваются разработчики. Изучение системы контроля версий и навыков работы с удаленными репозиториями подготовило к реальным условиям командной разработки и повысило уверенность в дальнейшей работе с Git и GitHub.

Список использованных источников

1. 100 самых ценных репозиториев GitHub URL: https://habr.com/ru/articles/453444/ (дата посещения 04.02.2026)

2. Программная инженерия. Интеграционный подход к разработке / Е. Л. Турнецкая, А. В. Аграновский. — Санкт-Петербург: Лань, 2023. – 216 с. – ISBN 978-5-507-46898-0. – Текст : электронный // Лань: электронно-библиотечная система. – (access : https://e.lanbook.com/book/352307).